

A systematic process for Mining Software Repositories: Results from a systematic literature review

M. Vidoni¹

The Australian National University, CECS School of Computing, Australia

ARTICLE INFO

Keywords:

Mining Software Repositories
Systematic literature review
Evidence-based software engineering
Guidelines

ABSTRACT

Context: Mining Software Repositories (MSR) is a growing area of Software Engineering (SE) research. Since their emergence in 2004, many investigations have analysed different aspects of these studies. However, there are no guidelines on how to conduct systematic MSR studies. There is a need to evaluate how MSR research is approached to provide a framework to do so systematically.

Objective: To identify how MSR studies are conducted in terms of repository selection and data extraction. To uncover potential for improvement in directing systematic research and providing guidelines to do so.

Method: A systematic literature review of MSR studies was conducted following the guidelines and template proposed by Mian et al. (which refines those provided by Kitchenham and Charters). These guidelines were extended and revised to provide a framework for systematic MSR studies.

Results: MSR studies typically do not follow a systematic approach for repository selection, and many do not report selection or data extraction protocols. Furthermore, few manuscripts discuss threats to the study's validity due to the selection or data extraction steps followed.

Conclusions: Although MSR studies are evidence-based research, they seldom follow a systematic process. Hence, there is a need for guidelines on how to conduct systematic MSR studies. New guidelines and a template have been proposed, consolidating related studies in the MSR field and strategies for systematic literature reviews.

1. Introduction

During the past decade, collecting and analysing data from open-source repositories of software projects has gained relevance in empirical software engineering research [1]. This field, known as Mining Software Repositories (MSR), analyses and cross-links data to uncover compelling and actionable information about software systems [2]. In MSR-based studies, researchers select repositories that fit specific criteria, extract data from them and analyse the data to obtain *evidence* to answer their research questions [3].

As a result, MSR is a flexible methodology, as it allows the empirical exploration of a range of questions. Example areas are software evolution, developers' networks and characterisation, bug prediction, effort estimation, among others [4]. Therefore, the popularity and acceptance of MSRs have grown in recent years, increasing the interest in this research area [5].

Since MSR-based studies are considered evidence-based research [2], having guidelines to produce structured and reproducible studies is critical. However, though meta-studies have focused on

different aspects of MSRs, such as the validity of some sources [6], or the usage of the data produced [7], there are no frameworks or methodologies to guide the entire process. This paper is motivated by the need to improve the reliability of MSR studies, understand how repositories are selected and define a systematic process for mining software repositories.

The contributions of this study are:

- Assess the current practice of conducting MSR-based studies in Software Engineering (SE) through a systematic literature review (SLR).
- Identify gaps in MSR-based studies and compare their process to that of SLRs (as defined in [8]).
- Centralise the findings to propose guidelines for systematic MSRs that support an unbiased aggregation of empirical results. These are derived from established SLR practices for SE [8,9] and supported by results from this SLR and previous related works by other authors.

E-mail address: melina.vidoni@anu.edu.au.

URL: <https://melvidoni.rbind.io/>.

¹ Corresponding Author.

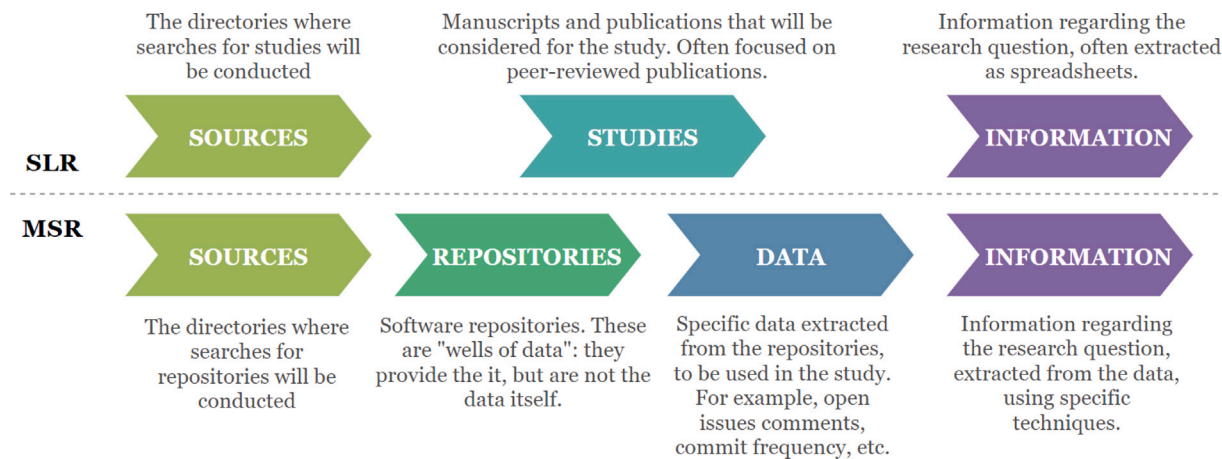


Fig. 1. Comparison of SLRs and MSRs steps.

The implication of this paper for both practitioners and researchers is the availability of a thorough process consolidating multiple findings related to a method for conducting MSRs. It is strengthened by further enabling the inclusion of MSRs as part of the evidence-based software engineering (EBSE) [10,11] paradigm by supporting the generation of replicable, comparable MSRs that can be applied across multiple domains or in large-scale studies.

The remainder of the paper is structured as follows. Section 2 presents background concepts and related work and outlines the motivation for this study. Section 3 explains the research method used for the SLR, while Section 4 presents the results. Section 5 describes the proposed guidelines for MSR-based studies. Finally, Section 6 discusses the implications of this proposal and Section 7 concludes.

2. Background, related work and motivation

This Section describes the background concepts used in this paper. The concept of evidence-based research in SE is defined, and the main differences with MSRs are analysed. The relevant secondary studies investigating MSR-based research are examined, and the motivation for this work is discussed.

2.1. MSR, SLR and mapping studies

Systematic mapping studies (SMSs) and SLRs are two popular types of *evidence-based* SE (EBSE) research. While SMSs are used to structure a research area, SLRs focus on gathering and synthesising evidence. In particular, Kitchenham and Brereton [8] compared the different characteristics of the process of SLRs and SMSs.

The application of EBSE involves asking answerable questions, finding the best evidence and critically appraising it, applying the evidence, and evaluating their performance [10]. The general goal of EBSE is "to provide the means by which current best evidence from research can be integrated with practical experience, and human values, in the decision-making process regarding the development and maintenance of software" [11]. As a result, it is possible to align traditional EBSE steps (i.e., those used in SLRs and SMSs) with the consensus of what constitutes an MSR study: both involve extracting evidence from different places (i.e., studies or repositories) to uncover useful patterns and information. Fig. 1 illustrates the critical processes of SLRs (top) compared to those of MSRs (bottom).

The goal of an SLR is to capture relevant studies in the search without bias and not exclude relevant evidence during the selection of evidence. SLRs are conducted by systematically establishing *where* to find material to analyse and what makes an existing *study* 'on-topic' for a research project (i.e., whether it should be considered or not) and methodically extracting data to produce information [12]. These

three 'parts' of the process translate to sources, studies and information, respectively.

Despite having conceptual similarities, MSRs also present important differences with SLRs and SMSs:

1. The *sources* of an SLR/SMS are the libraries (e.g., ScienceDirect and IEEE Xplore) used to find the 'material' to analyse. In MSRs, the *sources* can be of different types, such as platforms for hosting version control or bug-reports projects, mailing lists or even Stack Overflow. Therefore, Fig. 1 uses the word 'sources' instead of 'libraries'.
2. An SLR or SMS is more straightforward, as the 'elements' to be analysed are often the research papers found in the sources.² However, MSRs collect *repositories* (elements to be studied, such as projects in GitHub), but later need to extract specific *data sources*. For example, given two MSRs with the same sources and repositories, one may analyse only the GitHub issues, while the other may focus on the Readme files.
3. In MSR, the analysis for information can only begin after the data has been collected (as 'data' in Fig. 1). Moreover, the analysis process is usually manual in SLRs, while MSR usually automates it, especially in large-scale studies.

Currently, there are guidelines for conducting SMSs studies [12,13] and SLRs studies in SE [14]. Although both have been updated over time, SLRs guidelines were revisited by Kitchenham and Brereton [8] and structured into a template by [9]. This work extends that template to provide a systematic process for MSR-based research.

2.2. Related secondary studies

Several secondary studies, either SMSs or SLRs, have been conducted for MSR-based research. This Section discusses those more closely related to this work. Some of those concepts are integrated into the MSR Guidelines presented in Section 5.

Researchers have carried out several SMSs on MSR-based research. Among those, some relevant studies are:

- Farias et al. [15] mapped MSRs to collect the goals of the software analysis (such as purpose, focus and object of analysis), including data sources, evaluation methods, and tools. They were focused on the purpose of the studies and not the process behind the research. They mainly identified common data sources and types of data being extracted and mapped studies in work areas. Their dataset is publicly available.

² The word 'often' is used since, occasionally, SLRs/SMSs include other non-academic studies, such as patents or technical reports.

Table 1
GitHub perils identified by [6], included in the MSR guidelines.

Related to	Peril	Definition [6]
Project	(P1) Repository is not necessarily a project.	Repositories can be ‘base’ (the original source) or forks. This peril reflects the possibility of analysing a forked repository that contains incomplete data.
	(P2) Most projects have low activity.	In GitHub, the majority of commits and activities are concentrated on a few projects. Studying only these will produce non-generalisable results.
	(P3) Most projects are inactive.	Related to the above, most projects tend to be inactive, with few issues and sparse commits.
	(P4) Many projects are not software development.	Repositories tend to be used for other purposes, such as storage and website hosting. This usage cannot be reliably determined from project metadata.
	(P5) Most projects are personal.	Almost two-thirds of projects have a single committer, and almost 90% have fewer than three.
	(P6) Many active projects do not use GitHub exclusively.	Development projects often have mirrors (other repositories) or other systems, such as mailing lists and forums, making GitHub provide only partial information.
	(P7) Few projects practice pull requests.	Using pull requests is not widespread due to the personal nature of most repositories.
Pull requests	(P8) Merges only track successful code.	Merge commits require additional work to obtain the original commits, mainly if the merge is squashed.
	(P9) Many merges PR appear as non-merged.	If a project is only merging through <code>git</code> , then GitHub leaves these PRs as un-merged, even when they are not.
Users	(P10) Not all is activity due to registered users.	Many developers can be associated with the same email or pushing to a mirror, thus muddling the user-related information.
	(P11) Only users’ public activity is visible.	Private repositories cannot be mined as they are not accessible. The users’ activity in private repositories is hidden and cannot be accessed.
GitHub	(P12) API does not expose all data.	Among other problems, not all events can be retrieved through the API (only the last 300), not all of them have a timestamp, and it is not possible to track renamed, deleted or private entities.
	(P13) GitHub is continuously evolving.	GitHub often changes its features and interfaces, occasionally doing so silently. This may affect applicability and access.

- Dong et al. [5] studied the state-of-the-art in terms of research topics, data sources, type of data being extracted and the mining techniques and tools applied. They focused on approaches used to extract information from the mined data but not on the structure of the studies. They mapped them into research topics, identifying popular sources and linking them to mining tools.

SLRs have also been used to assess MSR-based research:

- Güemes-Peña et al. [16] conducted a review and provided a discussion on MSR studies, identifying papers using machine learning approaches to analyse the obtained data. This study focused exclusively on literature published in the MSR Conference. As a result, they classified selected papers into six topics, discussed research-based exclusively on the ‘Challenge Datasets’ provided by the MSR Conference and briefly discussed research opportunities.
- Chaturvedi et al. [17] examined papers with experimental analysis to identify datasets, techniques and tools used or developed in that literature set. It reviewed papers from the MSR Conference only, classifying them by tool type, dataset and queried repositories. Although the study evaluated the origin of data, it did not address its quality, selection or data extraction processes.
- Tripathi et al. [18] divided papers into two categories, depending on their reliance on open or closed-source software. They also reviewed papers from the MSR Conference exclusively. Although they assessed the origin of the data, they did not examine its selection, analysis, or threats to the validity of the studies.
- Finally, Hassan [19] completed an unstructured literature review and discussed the history of the MSR field, with an emphasis on achievements and techniques. The study did not address data selection or parsing, and classified research as either creating techniques or approaches for mining. This study also defined six areas of work and discussed future trends.

Several elements differentiate these works from the current proposal. First, although they addressed the MSR field, none of these studies focused on structuring the processes used for repository selection, data extraction and information mining. Second, two articles addressed threats to the validity of MSR-based studies but did not provide an approach for systematic MSR research. Finally, they did not compare the MSR approach to existing systematic methodologies

commonly used in Empirical Software Engineering, such as literature reviews and mapping studies. Therefore, as its main contributions to the literature, this paper covers these three ‘gaps’.

Two other papers are particularly relevant for the study in this manuscript, and their results were embedded in the proposed Guidelines (see Section 5). As such, the following subsections discuss their suggestions in more detail.

2.2.1. Scientific Ws

Vial [20] presented a position paper focused on the quality of *digital trace data* (DTDs). These are “digital records of activities and events that are produced, stored and retrieved using information technologies” [20]. In other words, this is the data that can be extracted from repositories.

On their work, Vial [20] discuss the ‘Seven Ws’ of scientific articles and relate them to the quality of DTD. They provide an analysis of how each W is relevant to DTD. The seven Ws are: *what* (what is captured in the data), *when* (timing for the collection), *where* (sources where repositories for data extraction are located; see Fig. 1), *how* (process to collect data from repositories), *who* (individuals involved in the collection), *which* (instruments or artefacts used to collect the data) and *why* (reasons or goals for collecting the data).

Though the paper provides no formal guidelines or frameworks, it addresses important issues on datasets disclosure and quality. The Guidelines proposed in this manuscript include the ‘seven Ws’ throughout the template and process.

2.2.2. Perils of GitHub

Kalliamvakou et al. [6] studied GitHub repositories, identifying threats that may affect the quality of the data or the analysis done with it. Despite focusing only on GitHub and not providing a structured approach for discussing threats to the validity of MSR-based studies, the work reveals thirteen ‘perils’ that may affect the quality of MSR studies along with considerations to minimise them. Table 1 summarises and numbers the perils, providing a brief definition to simplify referencing them later in the guidelines.

2.3. Motivation

Many factors motivated this work, derived from the previous analysis of related works.

Although the existing guidelines for SLRs/SMSs are thorough and widely accepted in EBSE [8,12,13,21,22], they require modifications to be used in MSR-based research. This is due to several intrinsic characteristics of MSRs. As discussed in Section 2.1 and seen in Fig. 1, MSRs require additional effort to acquire the ‘data sources’ since a ‘repository’ can provide multiple data sources (e.g., source code, issues, wikis). Although some data sources are similar (i.e., issues are somewhat similar regardless of being sourced from BitBucket or GitHub repositories), most require special consideration, which are not part of the existing SLR/SMS process. Moreover, MSRs can be large-scale, either fully or partially automated; this is not covered in the existing guidelines for SLR/SMSs.

Additionally, as highlighted in Section 2.2, the existing related works have only covered different parts of the process of MSR-based studies. They have mapped common topics of research to the type of data sources [15,16] or open/closed software [18], identified which techniques and tools are usually selected [17,23], studied how the field evolved [19] and which approaches are commonly taken [5]. Other meta-studies have also assessed the effectiveness of MSRs [24], the quality of traced data [20], the validity threats of specific sources [6] and how the information produced is being used [7]. Nonetheless, **to the author’s knowledge, no approach amalgamates this knowledge into a unified, consistent EBSE process for MSR-based research.**

Consequently, this study aims to simplify the adoption of systematic MSRs by consolidating the above information and providing an easy-to-follow template.

3. Method

This Section covers the method for the SLR. It includes research questions, structuring the search study, and data extraction. The results are discussed in Section 4.

3.1. Research questions

The goal of this study is to determine how MSR studies have selected their sources, repositories and data, and to identify research steps. The following research questions (RQs) were pursued:

- RQ1. Are papers based on MSR using specific, well-defined steps and processes to select repositories and data extraction?
- RQ2. Which repository sources are used, and which data is extracted?
- RQ3. Which conditions are being imposed during repository selection to define a systematic process?
- RQ4. Does MSR-based research discuss threats to validity, regarding repository selection?

This information is combined with existing best practices and guidelines to conduct SLR in SE and propose and develop specific Guidelines to generate systematic MSR-based research.

3.2. Search study

This secondary study focused on primary studies published in English in any peer-reviewed venue. Keywords were derived from the search statement and formed into sets related to the type of study, the sources and the repositories. Overall, the search string was defined as (*mining software repositories OR repository mining OR msr OR software repository selection*) AND (*github OR bitbucket OR gitlab OR stackoverflow*). All searches were performed in IEEE Xplore, ACM, ScienceDirect and Springer Link.

A manual search was executed, and the results were stored in Zotero, a reference management tool. Zotero was used to remove duplicates and manage the large number of references. The inclusion and exclusion criteria were defined as follows:

- **Inclusion Criteria:** The manuscript must be oriented to software engineering, being an MSR-based study. The study is not required to mention ‘mining software repositories’, but it must be a peer-reviewed journal article, a book chapter or a conference paper (this included workshops and co-located events, if and only if they were peer-reviewed). Additionally, it must be a full paper (i.e., not an extended abstract, poster, short paper, emerging results or positions).
- **Exclusion Criteria:** Manuscripts were excluded if published before 2005 or were secondary or tertiary studies of the MSR field. MSR-based studies were excluded if they: used a single repository, employed an existing dataset without explaining how it was obtained, were conference versions of extended journal articles (the latter was included) or addressed a source (e.g., GitHub) as a platform. In particular, studies using a single repository were discarded as the selection process was commonly not systematic and done ad hoc based on a specific quality (e.g., size, antiquity number of committers).

This SLR employs two key concepts: *sources* and *repositories*. The former refers to ‘where’ a search is conducted (e.g., GitHub, BitBucket, StackOverflow), while *repositories* are the element to be studied (e.g., a project in GitHub, a thread in StackOverflow). Roughly, they can be, respectively, compared to libraries and studies in an SLR or mapping study. It is worth noting that the SLR excluded manuscripts focusing on a single repository but not those using a single source (e.g., mining only from GitHub).

The search, selection and filtering process of this SLR is summarised in Fig. 2, from left to right. All studies obtained from the systematic search were collected in a Zotero library, and its integrated tools were used to remove duplicates automatically. Then, a *first screening* excluded articles after reading titles, abstracts and keywords only, while the *second screening* involved reading the full text and completing a quality assessment (in terms of sections, disclosure of steps and methodology detail). A total of 146 studies were selected. Each article was reviewed by a single author, which poses a threat to the reliability of the study. A forward snowballing approach was applied as a minimisation strategy in data collection (right-most step in Fig. 2), further discussed in Section 4.5.

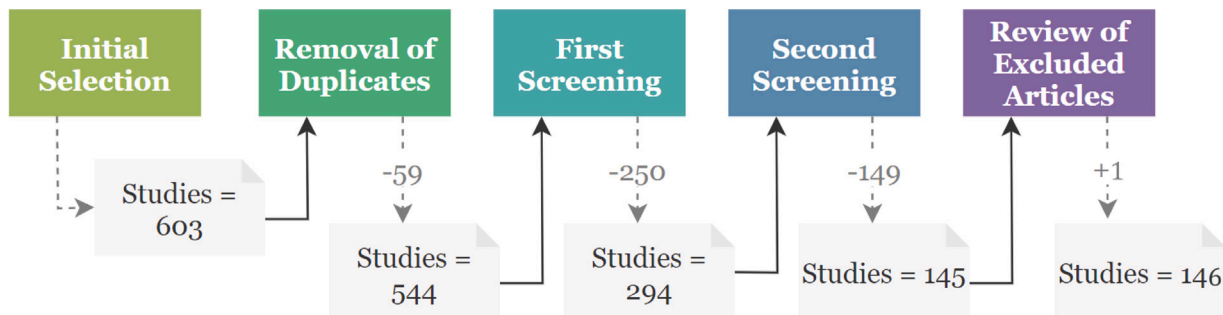


Fig. 2. Number of included articles during the study selection process.

Table 2

Template of the data extraction form.

Data item	Meaning	RQ
Study ID	An identifier for the article.	
Title	The title of the article.	
Year	The year of publication of the article.	
Venue	If the article was published in a journal or at a conference. Venue names were not recorded.	
Selection process	If the article details how some repositories were selected, the reasoning behind the sources and the inclusion and exclusion criteria.	RQ1
Selection steps	If the paper clarifies the steps followed to select the repositories. This includes, for example, filtering by dates, any possible tools used, if authors had a specific task and how they dealt with sources' limitations.	RQ1
Systematic selection	The selection can be systematic (predefined steps and filtering after selection), random (defines criteria and randomly download matching repositories until filling a sample size), unknown (not detailed in the manuscript), fixed (prefixed manual selection) or not systematic (merely downloading all repositories).	RQ1
Selection type	If the repositories were manually selected for some reason (i.e., size, popularity, activity, and so on), instead of generally downloaded according to inclusion and exclusion criteria. It can be 'predefined' or 'general'.	RQ1
Selection justification	Only considered for 'predefined' selections. Evaluates if the manuscript justifies the repositories selection or not.	RQ1
Mined sources	What sources, as defined in Fig. 1, were used in the manuscript.	RQ2
Sample size	The number of repositories that were collected. Empty if the data was not available in the manuscript.	RQ2
Content-type	What type of data was required that justifies the selection of the sources. Data is defined as in Fig. 1.	RQ2
Data extraction	If the manuscript details the steps required to extract the data from the repositories. It includes the steps and possible tools used for this, storage methods, and other relevant information. Dates of extraction may also be relevant if the repositories were not downloaded during selection.	RQ1
Selection biases	Threats to validity or biases about the selection of repositories only. The manuscript needs to discuss the selection process' issues to be counted as a 'yes'.	RQ4
Data biases	Threats to validity or biases related to the extraction of data from repositories. It can be related to tools, processes, dates discrepancies, among others.	RQ4
Filtering criteria	Which criteria were used to filter repositories.	RQ3

3.3. Data extraction

Table 2 was developed to extract data from the identified primary studies; for clarity issues, it also provides a brief definition of the goal of the field and the research question it addresses.

Some of the related works discussed in Section 2.2 already investigated research questions about the overall process of MSR studies. These are: (a) which repositories are more commonly mined, concluding that about 90% of MSR-based studies use open-source software (primarily focusing on Eclipse, Linux and Apache [18]) and (b) an in-depth analysis of data extraction protocols and techniques [19]. Therefore, since these two topics were already covered in previous works, the data extraction template did not include information related to them, instead opting to adopt the results of those manuscripts when creating the Guidelines (see Section 5). Note that the extraction implied reading the entire manuscript and inspecting its appendices (if any).

The extraction process was conducted in two rounds to minimise researcher bias [25]. In the first round, the author derived the data, and an independent researcher analysed a random subset of results; after that, both researchers met and discussed discrepancies and feedback on the classification. In the second round, the author revised the extraction using that feedback, and the independent researcher analysed a different random subset afterwards. In this case, there were no discrepancies or additional suggestions.

4. Results of the SLR

The following Section presents the results of the SLR.

Fig. 3 shows the number of MSR-based studies identified within 2005–2020 that fit the inclusion criteria. The prevalence of conference and workshop papers (both labelled 'Conferences') is due to the highly respected MSR Conference, co-located with the International Conference on Software Engineering (ICSE). The interest in this type of research increased considerably by 2017, but the numbers dropped slightly afterwards; note that 2020 was only partially included (up until May). Besides the surge of interest, another potential reason for the popularity of this type of research is the increased availability of public software repositories to mine. This rise in publication numbers indicates that MSR-based studies are considered highly relevant by the SE research community.

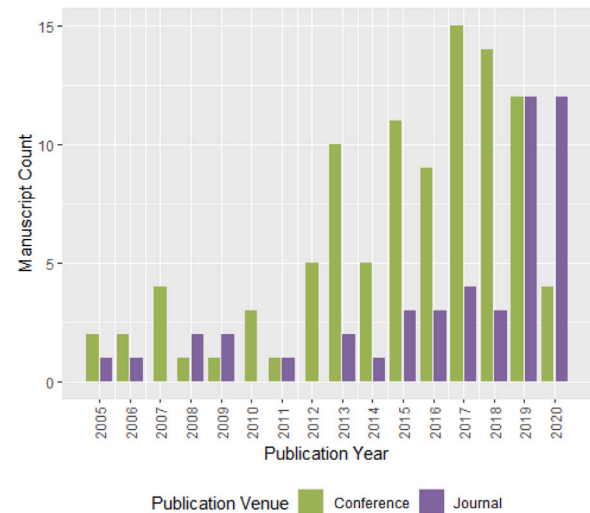


Fig. 3. Manuscripts included after the selection process.

4.1. Selection processes (RQ1)

This section analyses the data obtained from the manuscript by considering the conversion from *repositories* to *data* and to *information* happening in MSRs (see Section 2.1 and Fig. 1). Table 3 details the number of manuscripts that describe repository selection steps, and data extraction steps, in terms of their previous description of a repository selection process.

Approximately 37% of the manuscripts do not address the selection process of repositories and do not detail the selection steps. Thus, there is no disclosure about why the sources were selected, no explicit inclusion/exclusion criteria and no description of the protocol followed to select repositories (e.g., the time taken to complete and the role of the authors) or how they dealt with sources' limitations (e.g., GitHub's query limit).

Nonetheless, authors are prone to describe the process and steps performed to extract data from the repositories. Almost 70% of the

Table 3

Number of manuscripts that describe repository selection and data extraction, if they previously outlined the selection steps as an overview.

Describes...?		Outlined selection steps?	
		Yes	No
Data Extraction	Yes	77	25
	No	15	29
Repository Selection	Yes	49	0
	No	43	54

manuscripts had a section dedicated to the dataset generation.³ Furthermore, from the total sample, approximately 17% of papers did not describe the repository selection process but elaborated on the data extraction.⁴

In terms of selection, Fig. 4 shows that only approximately 30% of manuscripts worked with large, predefined repositories. The findings here correspond with previous works indicating Eclipse, Linux and Apache as the most used cases [18]. However, most of those studies (approximately 72%) do not justify the reasoning behind their fixed selection processes (middle bar, class *No*). The remainder of manuscripts do so by citing popularity, size and project life (middle bar, class *Yes*).

Only 17% of the manuscripts presented a systematic selection process. Approximately 28% clarified the inclusion/exclusion criteria but did not explain how collection limits were imposed when compared to ‘downloading everything’ (represented in class *No*) or ‘random sampling’ (represented as *Random*). Random sampling can be considered systematic if it is a collection boundary imposed due to computational or human limitations and if it follows selection criteria.

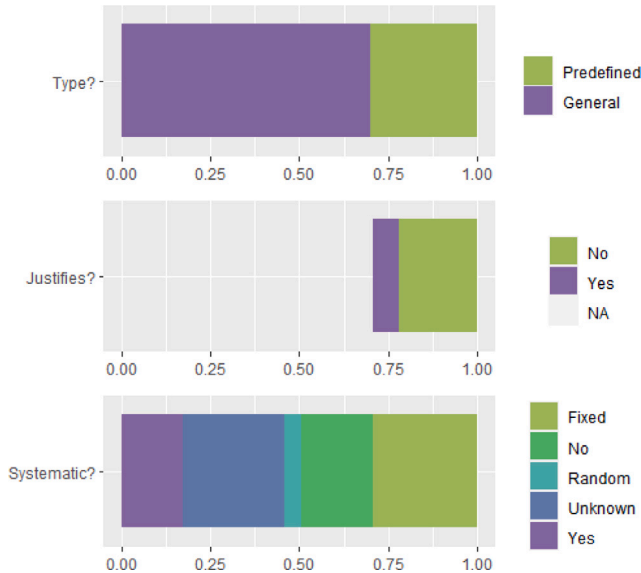


Fig. 4. Proportion of manuscripts for each selection characteristic. Only studies that worked with predefined repositories are accounted for in the ‘Justify’ bar.

4.2. Repository sources (RQ2)

From the selected manuscripts, about 10% used multiple sources in their study (e.g., combining data from StackOverflow and GitHub), while the rest focused on a single source. By far, the most common source was GitHub (67% of papers), followed by SourceForge (for older studies, 8%) and Bugzilla (7.5%).

Fig. 5 indicates the size of the repository sample per year of publication. Overall, 16% of the manuscripts did not mention the number of

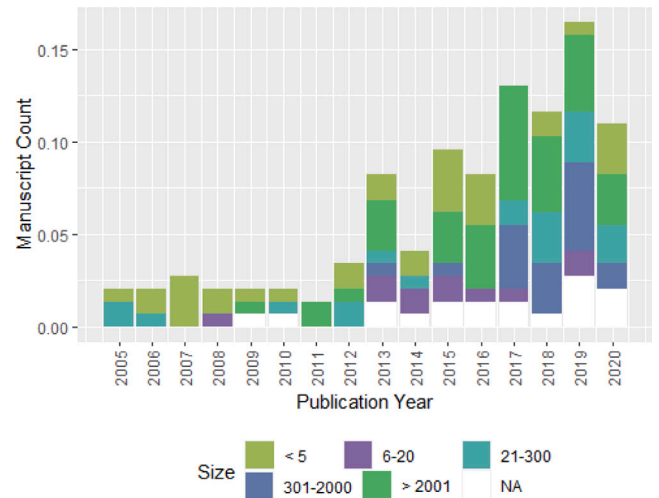


Fig. 5. Number of selected repositories per year of publication, in chronological order.

selected repositories; recently, this tendency has increased. Publications have leaned towards using larger sample sizes; this aligns with greater availability of more powerful hardware, better processing algorithms and a surge in the use of public repositories [17,23].

The selection of a source is determined by the type of data that needs to be collected; for example, bug data will require selecting Bugzilla or JIRA repositories, while version control could direct to GitHub, Azure or BitBucket. Around 27% of manuscripts had multiple targets, as a single source can provide several types of data (e.g., GitHub can provide *user data*, *version control data*, *metadata* and *source* at the same time). Fig. 6 illustrates this, indicating that version control data, source code and metadata are the most popular types of data. This finding aligns with the popularity of sources that were previously determined.

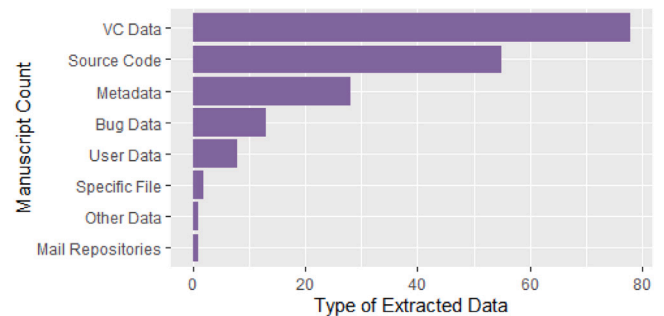


Fig. 6. Data that was extracted from the mined repositories.

4.3. Inclusion/exclusion criteria (RQ3)

Multiple manuscripts indicated one or more inclusion or exclusion criteria for the selection of repositories. The results are presented in Table 4.

The *programming language* is by far the most common criterion. After that, other popular options are searching for *specific files* (i.e., Readme files, Maven’s pom.xml and others) and filtering by *number of contributors* or by *specific version control events*. It is worth noting that the latter includes branching, committing, merging and reverting.

Nine manuscripts took into consideration whether the repository was original or forked from another, as indicated in the criterion *dependencies*. However, only five manuscripts considered how many times the target repository was forked (criterion: *number of forks*). Few papers mentioned the *project language* explicitly (i.e., natural language), even though several worked with natural language processing, exclusive to English.

³ Two cells, Data Extraction plus Outlined Yes.

⁴ Cell: Data Extraction/Yes, Outlined/No.

Table 4
Repositories' inclusion or exclusion criteria by manuscript.

Criterion	Manuscripts	Count
Programming language	[1,26–66]	42
Creation/Update dates	[31–33,35,47,66–83]	23
Popularity	[28,33,41,52,54,58,65,67–69,73,74,77,79,81,84–89]	21
Specific files	[27,36,43,54,57–59,66,85,86,90–95]	16
No of contributors	[34,35,44,66,68,73,79,82,85,88,90,96,97]	13
Specific VC events	[32,34,35,44,48,56,57,66,68,78,96]	11
Use of specific tools	[28,35,40,58,64,72,80,93,98,99]	10
Dependencies	[37,51,61,65,72,78,79,82,86]	9
Activity condition	[28,33,37,44,48,51,52,82,85]	9
Lines of code	[33,58,59,87,88,91,100]	7
No of forks	[44,52,60,68,75]	5
No of issues	[48,66,78,99]	4
Build tools	[54,57,86,95]	4
Search keywords	[27,38,59,83]	4
Application domain	[83,99,101]	3
No of related repositories	[85,96]	1
Available documentation	[28,102]	1
Comment quality/Number	[102]	1
Project language	[96]	1

4.4. Bias/validity discussion (RQ4)

A key to empirical studies is the discussion of validity threats. However, it is problematic that the vast majority of manuscripts (approximately 69%) do not address two cornerstone validity threats (see Fig. 7). The first one is biases or issues derived from the selection of repositories (including process, criteria, and others). The second is biases caused by the data extraction process or the data as provided by the sources. Only 13% of manuscripts discussed both cases, with the remaining 17.8% debating either one.

This is concerning considering that GitHub is the most popular source, and previous studies identified ‘perils’ that may influence the results obtained from mining it [6].

This analysis only considered discussions of validity threats or biases regarding the selection of repositories and the data extraction process. Many papers included a thorough discussion of validity in terms of the information analysis only, which is not included here.

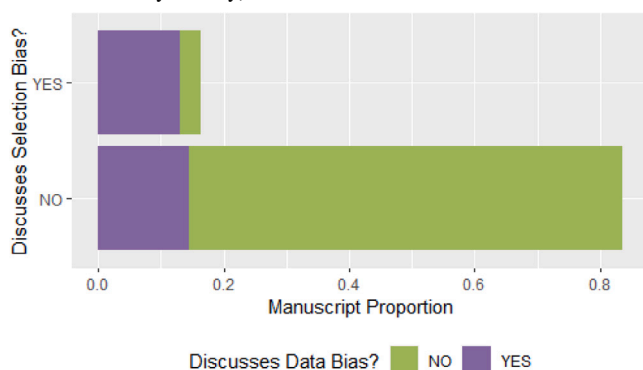


Fig. 7. Proportion of manuscripts that discuss bias in the selection of repositories or in the extraction of data.

4.5. Validity threats

This Section discusses the threats to the validity of this SLR before presenting the proposed guidelines for MSR-based studies. Theoretical validity is determined by the author's ability to capture what was intended, and confounding factors, such as biases and selection of studies, play an essential role.

Researcher bias may appear during the process of selecting and extracting data. The study was conducted by an individual author, which is the main threat to validity. Hence, some studies may have been missing. To mitigate this threat, the study identification was

evaluated through a reference set of articles, with the author applying a forward snowballing sampling approach, following existing guidelines [13]. This strategy was also used in a similarly structured systematic study [22]. The excluded papers were revised by an independent researcher (title and abstract only) and considered ‘noise’. ‘Noise’ papers contain keywords but work on different areas (e.g., ‘mining software’ papers related to new tools in the mining engineering area). This allowed the process to be conducted individually [103] since the independent review determined no bias in the selection process. This is illustrated in Fig. 2. A single new study was obtained, indicating that the overall conclusions of this literature review would not change.

Researcher bias is also a threat during extraction and classification stages and in the interpretation of results. This risk was mitigated by conducting an independent evaluation of the extracted data, as done in other studies [25]. After the author finished the extraction, an independent researcher randomly analysed a set of studies, comparing their outcomes to those of the researcher's; a follow-up discussion was used to analyse discrepancies and obtain feedback. The author revised the extraction using this feedback and followed up with a second round of analyses.

Regarding the selection, there is also a possibility that MSR-based studies do not contain keywords such as ‘mining’ or ‘repositories’. As a result, there is a threat that these studies are missed during the inclusion and exclusion phase. This study was conducted during early 2020; hence, only studies from the first third of the year are included in the analysis. Furthermore, 146 articles were identified, providing a sample of an acceptable size. Different venues are well represented, and the relevance of the MSR conference in this area is acknowledged.

There is a potential threat that activities may be overlooked, misunderstood, or incompletely reported as the analysis was based only on the manuscripts. This can affect the completeness of the classifications in terms of selection and extraction steps.

5. Guidelines for systematic MSR studies

The use of guidelines has proven advantageous in several areas, as they provide the rationale for structuring processes according to what has worked successfully for previous applications, environments, and people [104]. Therefore, the approach taken to generate the Guidelines presented in this Section is a modified version of Guidelines-Based Software Engineering (GSE) [105]. This modification was necessary as GSE is oriented to software components and not processes. Overall, the following steps were completed:

1. *Identifying business requirements and target artefacts:* This step involves identifying the domain to be studied and which guidelines

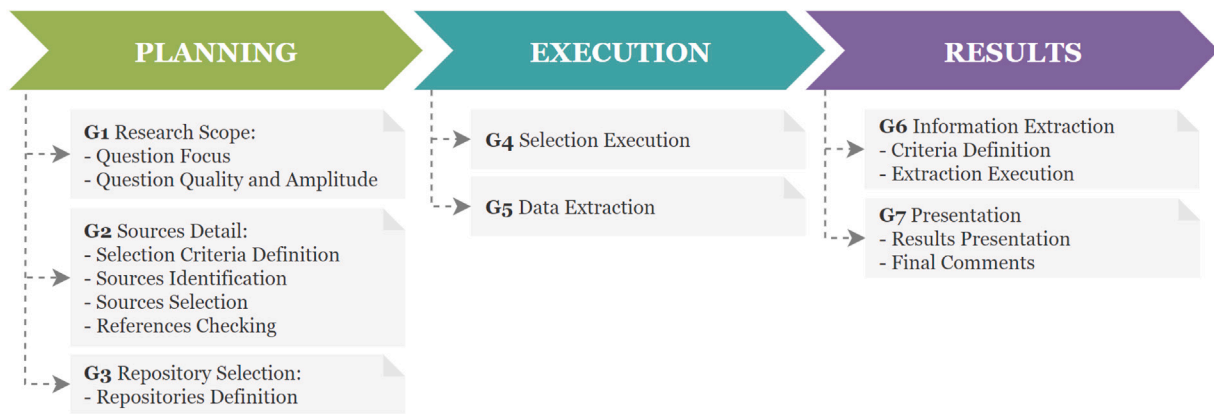


Fig. 8. Summary of the guidelines for systematic MSRs.

will be developed. It relates to the motivation and research goal of this work, which was discussed in Section 2.3. Regarding GSE, it covers the ‘setting of domain principles’ [105].

2. *Conducting a domain analysis and classifying best practices:* This step requires gathering domain knowledge about what was done in terms of theory and practice for the domain. The former is related to the existing SLR/SMSs methodologies and the EBSE paradigm, discussed in Section 2.1. This enables the identification of the differences and commonalities between the available theory and the target domain (MSR-based research). The study of the current practice in the existing domain was completed through the SLR of primary MSR-based studies, outlined in Section 3 and analysed in Section 4. Regarding GSE, this implied ‘data collection’ and ‘data analysis’ [105].
3. *Formulation, revision and refinement:* The last step was a cyclic process that consisted of ‘data analysis’ and ‘classification’ [105] to consolidate the results of the previous step. The starting point was the template proposed by [9] given that it “aggregates and formalises ideas regarding [...] guidelines for systematic review in SE proposed by Kitchenham [21]”; the update of 2013 [8] was also considered. From there, an iterative process of analysis and classification led to the re-organisation of the sections in terms of MSR-based steps, with the selection, filtering, and validity analysis extended in alignment to the ‘Seven Ws’ [20], and the perils of GitHub [6]. Multiple mitigation strategies and proposed steps were derived from the actions disclosed in the manuscripts considered in the SLR.

Noting that all elements presented in the manuscript were used for the elaboration of the Guidelines during Step (3):

- The related EBSE work discussed in Section 2.1 was used as the baseline since it is a sound approach that has been widely adopted for aggregating evidence-based data.
- The related secondary MSR studies (such as those on the ‘Seven Ws’ and the Perils; see Section 2.2) were incorporated as they provide complementary information to the findings of the SLR.
- The SLR conducted in Sections 3–4 aimed to: (a) define the phases relevant to MSR studies, (b) uncover common problem areas that should be further systematised to reinforce the replicability of studies, (c) identify which specific minimisation strategies were commonly used and (d) determine how to best present these points in the resulting manuscript. The explanations often include examples derived from the SLR.

The ‘evaluation of models’, aimed to assess the Guidelines in practice, was not included in this study. A thorough practical evaluation of the proposed Guidelines for Systematic MSR Studies remains future work.

The following subsections present the guidelines organised in the main phases of a systematic study: planning, execution, and results analysis. They are summarised in Fig. 8.

5.1. Guidelines: Planning

This phase aims to define the research objectives and the way the MSR study will be executed. This includes formulating research questions and planning the selection of sources and repositories. It is divided into three sections.

5.1.1. Planning: Research scope

The research objectives must be clearly defined by filling in the following items.

G1.1. Question Focus: ‘Defines the MSR focus of interest (i.e., the research objectives).’ [9].

G1.2. Question Quality and Amplitude: This Section should help define the research questions to clarify the context in which the MSR is applied. In particular, this must include:

- *Questions:* Research questions to be answered through the MSR study.
- *Intervention:* What is being observed in the context of the planned MSR. In particular, this will determine the perils (see Section 2.2.2) that may affect the selection of sources (if applicable), as well as limitations that may be imposed to counteract them.
- *Control:* Baseline or initial dataset already available to the researcher. It can be example (or suggested) repositories, data templates (i.e., indications of the targeted format) or information samples (from previous studies).
- *Effect:* Types of results expected at the end of the MSR study. This allows backtracking the steps from results to information, the data required for it, the repositories that host it and the sources.
- *Outcome Measures:* Metrics used to measure the effect. It depends on the type of effect to be achieved.
- *Population:* This defines the ‘group’ observed in the study. For MSR-based research, this implies the repositories that will be selected and, afterwards, mined to extract specific data to be used in the study.
- *Experimental Design:* Meta-analysis to be conducted, defining analysis methods to move from data to information and to interpret that result. It covers a “data cleaning and wrangling” section to address the sources’ perils (see Section 2.2.2) that cannot be contained through inclusion and exclusion criteria. This is detailed in Table 5.

The suggestions of Table 5, column ‘Action’ are drawn from those applied in different manuscripts selected for the SLR (see Section 4).

Table 5

Perils [6] and points of actions to be included in the ‘experimental design’.

Peril	Actions	Sources
(P10) Not all activity is due to registered users. (P12) Limited API data.	Email unification, clean up of contributors data. Use different API keys. Query many times.	All
(P6) Multiple sources.	Match them and filter data, remove duplicates, unify records.	
(P13) Evolving sources	Limit data analysis over periods to determine if it may affect specific results. Evaluate before and after a particular change.	
(P8) Merges only track successful code. (P9) Many merges PR appear as non-merged.	Mine git information, use other sources. Include heuristics to ‘clean up’ merge detection [6].	GitHub

5.1.2. Planning: Sources details

The objective of this Section is to select the sources where searches for software repositories will be executed. This Section addresses two of the ‘Seven Ws’ (see Section 2.2.1). The following items need to be completed.

G2.1. Sources Selection Criteria Definition: Defines criteria used to evaluate the sources by considering the characteristics that make them candidates for the MSR-based study. This is also related to the type of information to be used (see Fig. 6).

- **Definition of Repository Types:** It defines the types of repositories to select during the systematic selection. The type of data to be extracted should be considered (i.e., requiring the source code only would potentially rule out Bugzilla). This answers the ‘where’ [20]: some repositories may be online (i.e., GitHub), but others can be surveys or interviews used to cross data among domains (examples: [69,76,90]).
- **Statement of Purpose:** This answers the ‘why’, as it “provides the set of reasons or goals for collecting the data” and selecting sources [20]. It ensures a clear and well-justified link between the data collected, the reasoning for the sources selection, and the research questions.

G2.2. Identification and Description of Perspective Sources: This item concentrates on the selection of the sources for the MSR execution. It includes:

- **Sources’ Search Methods:** Describes how to search through the sources’ web engines; this may include using specific repository downloading/mining tools (e.g., GHTorrent or GitHub Archive). Constraints in the search (i.e., GitHub search return limit, or StackOverflow’s daily query limit) should be addressed here.
- **Languages of Choice:** It defines the languages in which the repositories or code need to be documented. It is relevant because if the study evaluates data depending on natural language (e.g., comments, commit messages, readme files), then researchers may need to constrain it to read it and apply processing tools (example: [96]). It should be consistent with the *Experimental Design*; for instance, by using a library of English stop-words when the study works with code comments written in Italian.
- **Sources List:** Initial source list in which the MSR-based study will be run. This selection should consider: a) the number of repositories that could be obtained and how suitable this is for the planned study, b) the quality of the data extracted from that source, c) terms and conditions that may limit the access to information, and d) other factors that might influence the source selection (e.g., future works, popularity, community).

G2.3. Sources Selection: If a source fits all the criteria, it must be included in the final sources list, presented in this Section of the protocol.

G2.4. References Checking: One or more experts should evaluate the sources list obtained from the previous item. This step may include testing and ensuring access privileges and data transfer quotas, checking documentation and talking to professionals and experts. It can include a peer-review step designed to minimise bias, conducted either internally (between members of the research group) or externally (with an independent expert or researcher). In case sources are modified, those changes need to be described in this item.

5.1.3. Planning: Repositories selection

Once the sources are defined, it is necessary to describe the process and the criteria for repositories selection and evaluation. It should address all the items described below.

G3.1. Repositories Definition: This item defines the way repositories will be selected. It considers the following:

- **Repositories Description:** Also answers the ‘what’ [20], by explaining the nature of repositories and why they can yield the required data.
- **Repositories Timing:** Related to the ‘when’ [20]. Even if performed with the same criteria, a repository selection can yield different results if executed at different times (e.g., the most forked repositories from 2010 may not be the same as those from May 2020). It is the date on which the repository selection and downloading occurred.
- **Inclusion and Exclusion Criteria Definition:** Presents the criteria by which repositories will be evaluated to decide if they must be selected or not. This is related to specific characteristics of the repositories that will be mined. Table 6 presents a list of criteria developed in consideration of criteria discovered in the SLR, matched to the perils (Section 2.2.2) as *limits* (potentially neutralises the peril), or as *affected by* (the peril may reduce the sample size further).⁵

Table 6

Inclusion and exclusion criteria for repositories, and their relationship to the sources’ perils.

Criteria	Limits peril	Affected by peril
Programming language	P4, P1	
Dates (between, or up to)	P13	
Team size (Contributors)	P5	P11
Version control events	P2, P7	P12, P13
Popularity		P11, P13
Specific files	P4	
Maintenance condition	P2, P3	
Libraries, Build tools	P4	P13
Repository size		
Dependencies	P1, P6	

- **Procedures for Repository Selection:** Describes the procedure by which the repositories will be obtained and evaluated according to exclusion and inclusion criteria (see point above). If the process has more than one stage (i.e., filtering steps), all of them must be described. According to what was found in the primary studies sourced for the SLR (see Fig. 4), it can be:

⁵ The matching between these concepts (perils and criteria, as ‘limits’ or ‘affected by’) was completed during Step 3 of the GSE elaboration, outlined in Section 5. The relationship was deducted by a detailed analysis of how each specific criterion was used in different manuscripts sourced for the SLR; specifically, this pertains to the findings of RQ3, described in Section 4.3. The primary studies used for this are listed in Table 4. Depending on the justification given by the authors of each manuscript, the criteria were listed as ‘limit to a peril’ or ‘affected by’ it.

- *Fixed*: Only specific, manually predefined repositories are used.
 - *Filtered*: A large sample is reduced after applying inclusion or exclusion criteria.
 - *Random*: The original sample (repositories, data or both) is too large to be handled adequately under current resources constraints (computational, human or both). An appropriate statistical method should be used to determine the sample size. It should also be filtered.
 - *Complete*: A minimal filtering is applied at commencement, and then all repositories in a query are downloaded and collected.
- *Search String*: Keywords used in the sources' search engines. This item should present "a set of logical expressions that combine keywords arranged in a way that the highest amount of relevant [repositories] is obtained from the search engine" [9]. In MSRs, many sources have different limitations regarding results provided by a search (e.g., GitHub only returns up to 1000 repositories per query). The following are example search tactics that can be applied:
 - Performing multiple searches in the same source, with the subsequent strings excluding repositories already collected. Example: [92].
 - Negating strings to reduce the time required for data gathering and to ensure a comprehensive sample. Example: [91], based on the proposal of [106].

5.2. Guidelines: Execution

After evaluating the planning, the systematic MSR execution can be initiated. The search should be executed in the defined sources, and the repositories should be assessed according to the criteria. After that, data extraction can commence.

5.2.1. Execution: Selection execution

This section continues the items described in Section 5.1.3.

G4.1 Selection Execution: This Section aims to register the selection process of repositories, reporting the results of the evaluation. The following items should be described:

- *Initial Repository Selection*: Researchers execute the search, and list all repositories for further evaluation. At this point, two options are available: a) only listing repositories to be used, or b) fully downloading the repositories. While (a) simplifies the need for storage space, a repository excluded during the search could be included on the extraction date due to users' activities (e.g., more commits, more watches, more downloads). Option (b) avoids this problem at the cost of storage space.
- *Repositories Quality Evaluation*: The quality of the repositories must be ensured since it can affect the results obtained. Filtering stages should be recorded, detailing how many repositories were discarded or added at each stage. Additional steps will depend on the selected source (i.e., the same quality criteria cannot be applied to StackOverflow posts and GitHub repositories). Overall, it should: a) ensure the sampling is not biased, b) allow results to be generalisable to some extent, and c) be capable of providing the data needed for the study, and d) use a justifiable external measurement of quality, relevant to the study, if applicable (e.g., popularity or forks in GitHub, installations in CRAN or PyPi). The following are negative examples (namely, they indicate an incorrect report): mining GitHub repositories only created in a particular country unless the study poses RQs related to the region (for point a), and if using GitHub's search, not explaining the entire search string and additional options used (for point b)).

- *Selection Review*: The selection of repositories must be reviewed to guarantee that no relevant repositories were eliminated. Independent reviewers may be helpful, and the results of this review must be recorded in this item.

5.2.2. Execution: Data extraction

This Section was generated by consolidating the steps usually reported when extracting data from manuscripts in SLRs [8,9], as well as the three data parsing steps discussed by Vial [20]. The latter are: extracting the source data, transforming it to a format fit for analysis (information extraction), and loading (saving it as a curated dataset).

G5.1. Data Extraction: This should describe the mining process to extract data from the repositories. It should include details for the three parsing steps, following the items below:

- *Data Inclusion and Exclusion Criteria Definition*: It is the criteria by which the data obtained must be extracted. This defines the type of data that is required. It must be consistent with the plans in *Experimental Design*, and be mined from repositories. It should include:
 - An explanation of the nature of the DTD and its qualities and how it relates to the question(s). This also answers the 'what' [20].
 - The dates of extraction and parsing, to also answer the 'when' [20]. As explained in previous sections, if the repositories and the data are selected and extracted at different times, results may be inconsistent (e.g., new issues can be created during this period, bugs' status can change). This is relevant as it affects the reproducibility of the study. This clarification could be simplified if the repository is downloaded during selection, and the data extraction is performed on the downloaded data, saved as a repository for future analyses (see the first item in Guidelines G4.1).
- *Extraction Execution*: It should 'describe the precise process(es) of data collection and mining' to answer the 'how' [20]; this includes mining algorithms, steps required such as crossing data between databases and cleaning heuristics to minimise perils [6]. Furthermore, it needs to "detail instruments or artefacts used in collecting the data" to complete the 'which' [20]; it embodies languages and tools used to run mining algorithms, pre-processing and structuring, the hardware involved in the process, and processing times.
- *Data Extraction Format*: Must standardise the way data is represented, creating forms to collect the data from the repositories. These vary depending on the data, the mining algorithms to extract and process it, and the databases employed to hold the data.

5.3. Guidelines: Results analysis

After executing the systematic MSR, results need to be summarised and analysed using the methods defined in the planning phase. It should include the extraction of information from the data (see Fig. 1) and the presentation of results.

5.3.1. Analysis: Information extraction

After extracting and organising the data, the researcher must start the generation of relevant information. The following items need to be described in this Section of the guidelines:

G6.1. Information Criteria Definition: Criteria by which the information obtained from the data must be evaluated. This should be consistent with the *Experimental Design*, and enable the *Effect* and *Outcome Measures*.

G6.2. Information Extraction Execution: Two different processes can be performed depending on the information to obtain. These methods are chosen during the *Experimental Design* and applied to analyse data and understand the complex relations between the obtained results.

- **Manual Process:** This is required when algorithms for information extraction require training before being applied to large datasets (i.e., supervised learning approaches). This manual process must describe:
 - Steps required to complete the manual assessment.
 - How the gold data (training and test sets) are defined and obtained, including sample size, as well as statistical procedures used to determine them.
 - Information extraction forms to standardise the format used to represent information. These will be used to train the algorithms during the automated process.
 - ‘Who’ [20] completed manual steps, potential threats to the validity of their work, how it was addressed, and reviewers included in the steps.
- **Automated Process:** The process used to generate information is defined. It includes details similar to the *Data Extraction* (guidelines G3.3): ‘how’ (to describe the precise processes to collect, cross and validate the information) and ‘which’ (to include tools employed during this process).

G6.3. Information Extraction Format: Standardises how information is represented. Similar to the data, this is influenced by the processing algorithms, the research questions, and the selected databases.

G6.4. Sensitivity Analysis: The robustness of results must be verified. This implies investigating uncertainties in the inclusion and exclusion of specific repositories and data. Sensitivity analysis is relevant when performing a meta-analysis.

5.3.2. Analysis: Results presentation

This Section aims to present the results of the study. It must describe the points below:

G7.1. Results Presentation: The results obtained from the analysis must be displayed in a way that facilitates the analysis. The format is constrained by the type of information extracted, as different approaches (e.g., graphs, tables, specific charts) may suit different types of information.

G7.2. Final Comments: This item presents the researchers’ final comments about the results. It should discuss the following items:

- **Mining Numbers:** Final quantities related to *data extraction* (e.g., the number of issues, commit messages). This includes only the data extracted from repositories to generate the information and not all the data contained in the repositories. *information* numbers, and its relationship to the data (e.g., size of the networks generated for each developer).
- **Search, Selection and Extraction Bias:** This must describe biases that can invalidate the results. Three types of threats must be addressed (see Fig. 7):
 - **Repository Bias:** It might exist if the selection of repositories influences the data extraction and, hence, the information in a transitive manner.
 - **Data Bias:** It can occur when there are threats to the validity of the data extracted from the repositories. It needs to address the perils that affect the selection criteria (Table 6), as well as those considered in the *Experimental Design* (Table 5). Table 7 provides more detail on how to address some of those perils.
 - **Information Bias:** Some algorithms can be biased or have inherent problems affecting the results. For example, there might be a limited amount of processing or algorithms’ constraints [107,108]. This is relevant as their “improper use can lead to biased results” [109]. This Section should be tailored for each MSR-based study, centred on a bibliography specific to the applied algorithms.

Table 7

Perils not constrained by inclusion/exclusion criteria, and how to address them as validity threats in an MSR-based study.

Peril	Threat discussion
(P8) Merges tracks successful code only	Potentially missing data, and its impact on the results. Steps completed to avoid this peril.
(P12) API does not expose all data (P10) Not all activity is due to registered users (P11) Only users’ public activity is visible	
(P9) PR merges may appear as non-merged	Potentially incorrect data and its impact on the results. Steps completed to avoid this peril.
(P13) Sources continuously evolving	Which changes were detected, and where was this information found. How this was compared to the repositories and data retrieved (e.g., which steps were followed). If the study falls within those dates, which steps were taken to unify or standardise the data.

6. Implications

The importance of the MSR field is increasing, supporting both software development practice and software engineering research alike [1]. Thus, this study has several implications for SE research.

Overarching, consolidated perspective. Although other secondary studies followed the MSR process, they covered different aspects without providing an overarching analysis (see the discussion in Section 2). As a result, this study was designed in consideration of existing research, targeting research questions that were not previously studied (see Section 3.3). First, this enabled identifying trends in the reporting of MSR studies that can be recognised as problematic since they lower the clarity of the process, of the data extraction, or of the results. Second, based on existing, comprehensive research results and thorough new analysis, the Guidelines proposed in this paper consolidate multiple results and provide a comprehensive, systematic process.

Support for Conducting MSRs. EBSE argued for a shift in research in SE, aiming to “provide the means by which current best evidence from research can be integrated with practical experience, and human values in the decision-making process regarding the development and maintenance of software” [10,11]. To guide this field, well-developed processes for conducting SLRs and SMSs enabled their usefulness and relevance in SE research [22,110]. MSRs are also EBSE (see Section 2) and require an overarching methodology for performing an unbiased aggregation of empirical results. In addition to both sources discussed above, the proposed Guidelines are derived from those used for SLRs, thus bringing together this research paradigm.

Note that the Guidelines produced in this study focus on the systematisation of data source identification, data gathering, reporting, threat analysis and traceability. However, the Guidelines do not cover other technical aspects of MSR, such as processes for storage modelling, data pre-processing, data cleaning, interpolation and transforms, model and analysis validation, and data provenance. Though it is expected to extend the Guidelines in future works, most of these elements are determined by the data being extracted and the research questions. Thus, they should be completed using relevant frameworks and methodologies, according to the state-of-the-art in each area.

Replicable and comparable MSRs. As part of EBSE, MSRs need to be critical, outlining how data was obtained and aware of risks and validity threats. The provided Guidelines cover this and provide a template to simplify the process. By following it, the results of MSRs can be compared and replicated, allowing extending studies to other domain areas (e.g., other programming languages or sources) to diversify

knowledge. Likewise, it also supports the continuation and expansion of existing works (e.g., by conducting large-scale replicas), which is not possible if the process is not clear.

7. Conclusion

This Systematic Literature Review (SLR) identified studies that performed Mining of Software Repositories (MSR) and evaluated them in terms of descriptions of repository selection, data extraction, discussion of biases, and criteria defined to filter repositories. The answers to the research questions of the SLR are presented below:

- **RQ1, Selection Processes:** About 37% of manuscripts do not address the repository selection process and do not detail selection steps, even though close to 70% of the manuscripts had a section dedicated to the generation of the dataset. Furthermore, only 17% exhibited a systematic selection, with approximately 28% describing inclusion or exclusion criteria, without clarifying recollection limits or steps.
- **RQ2, Repository Sources:** Approximately 16% of the manuscripts did not clarify the number of selected repositories, with larger sample sizes being used in recent years. Furthermore, most research focused on sources that provided data related to version control or source code.
- **RQ3, Criteria:** Multiple manuscripts indicated inclusion and exclusion criteria for the repositories. The most popular criteria are programming language, creation or update dates, and repository popularity.
- **RQ4, Bias Discussion:** Only 13% of manuscripts discussed biases related to both the repository selection and the data extraction processes; 17.8% addressed either one or the other. This did not include discussions related to the validity of the analysis.

MSR is a field that is continuously increasing in relevance as more and more public repositories are continuously created. However, there are no systematic guidelines to conduct this type of research. Furthermore, the SLR indicated that most MSR-based studies do not report a systematic research method. This motivated the proposal of guidelines for systematic MSR studies.

Due to some similarities with SLRs processes, the Guidelines proposed in this paper are based on those provided for SLRs [8,9]. It also incorporates studies by other authors related to validity threats in MSRs [6,20], as well as the results from the SLR of this manuscript. Therefore, valuable information on how MSR studies can be conducted systematically is concentrated in this comprehensive framework.

The Guidelines need to be validated in practice. As future work, this implies conducting several MSR-based studies for different domains and purposes to test the versatility of the guidelines. With this, it will be possible to refine the guidelines further.

CRedit authorship contribution statement

M. Vidoni: Conceptualisation, Methodology, Conduction, Writing.

Declaration of competing interest

No author associated with this paper has disclosed any potential or pertinent conflicts which may be perceived to have impending conflict with this work. For full disclosure statements refer to <https://doi.org/10.1016/j.infsof.2021.106791>.

Acknowledgement

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Appendix. Included studies

The following studies are included: [1,26–75,78–102,111–178].

References

- [1] F. Trautsch, S. Herbold, P. Makedonski, J. Grabowski, Addressing problems with replicability and validity of repository mining studies through a smart data platform, *Empir. Softw. Eng.* 23 (2018) 1036–1083, <http://dx.doi.org/10.1007/s10664-017-9537-x>.
- [2] A.E. Hassan, The road ahead for Mining Software repositories, in: 2008 *Frontiers of Software Maintenance*, 2008, pp. 48–57.
- [3] M. Felderer, F. Jeschko, A process for evidence-based engineering of domain-specific languages, in: *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018*, in: EASE'18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 169–174, <http://dx.doi.org/10.1145/3210459.3210479>.
- [4] Y. Kamei, A. Zaidman, Guest editorial: Mining software repositories 2018, *Empir. Softw. Eng.* (2018) 1–3.
- [5] L. Dong, B. Liu, Z. Li, O. Wu, M.A. Babar, B. Xue, A mapping study on mining software process, in: 2017 24th Asia-Pacific Software Engineering Conference (APSEC), IEEE, Nanjing, 2017, pp. 51–60, <http://dx.doi.org/10.1109/APSEC.2017.11>, URL: <http://ieeexplore.ieee.org/document/8305927/>.
- [6] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D.M. German, D. Damian, An in-depth study of the promises and perils of mining GitHub, *Empir. Softw. Eng.* 21 (2016) 2035–2071, <http://dx.doi.org/10.1007/s10664-015-9393-5>.
- [7] Z. Kotti, D. Spinellis, Standing on shoulders or feet? the usage of the MSR data papers, in: *Proceedings of the 16th International Conference on Mining Software Repositories*, in: MSR '19, IEEE Press, Montreal, Quebec, Canada, 2019, pp. 565–576, <http://dx.doi.org/10.1109/MSR.2019.00085>.
- [8] B. Kitchenham, P. Brereton, A systematic review of systematic review process research in software engineering, *Inf. Softw. Technol.* 55 (2013) 2049–2075, <http://dx.doi.org/10.1016/j.infsof.2013.07.010>, URL: <http://www.sciencedirect.com/science/article/pii/S0950584913001560>.
- [9] P.G. Mian, T. Conte, A.C.C. Natali, J.C. de Almeida Biolchini, G.H. Travassos, A systematic review process for software engineering, in: *Experimental Software Engineering Track (ESELAW)*, 2005, pp. 1–6, URL: <https://api.semanticscholar.org/CorpusID:17417820>.
- [10] T. Dybå, G. Bergersen, D. Sjøberg, Evidence-based software engineering, in: T. Menzies, L. Williams, T. Zimmermann (Eds.), *Perspectives on Data Science for Software Engineering*, Morgan Kaufmann, Boston, 2016, pp. 149–153, <http://dx.doi.org/10.1016/B978-0-12-804206-9.00029-5>, URL: <http://www.sciencedirect.com/science/article/pii/B9780128042069000295>.
- [11] B.A. Kitchenham, T. Dyba, M. Jorgensen, Evidence-based software engineering, in: *Proceedings of the 26th International Conference on Software Engineering*, in: ICSE '04, IEEE Computer Society, USA, 2004, pp. 273–281.
- [12] K. Petersen, N.B. Ali, Identifying strategies for study selection in systematic reviews and maps, in: 2011 International Symposium on Empirical Software Engineering and Measurement, 2011, pp. 351–354, <http://dx.doi.org/10.1109/ESEM.2011.46>.
- [13] K. Petersen, R. Feldt, S. Mujtaba, M. Mattsson, Systematic mapping studies in software engineering, in: *Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering*, in: EASE'08, BCS Learning & Development Ltd., Swindon, GBR, 2008, pp. 68–77.
- [14] P. Brereton, B.A. Kitchenham, D. Budgen, M. Turner, M. Khalil, Lessons from applying the systematic literature review process within the software engineering domain, *J. Syst. Softw.* 80 (2007) 571–583, <http://dx.doi.org/10.1016/j.jss.2006.07.009>, URL: <http://www.sciencedirect.com/science/article/pii/S016412120600197X>, Software Performance.
- [15] M.A. Farias, R. Novais, M.C.C. Júnior, L.S.P. da Silva Carvalho, M. Mendonça, R.O. Spínola, A systematic mapping study on mining software repositories, in: *Proceedings of the 31st Annual ACM Symposium on Applied Computing*, in: SAC '16, Association for Computing Machinery, Pisa, Italy, 2016, pp. 1472–1479, <http://dx.doi.org/10.1145/2851613.2851786>.
- [16] D. Güemes-Peña, C. López-Nozal, R. Marticorena-Sánchez, J. Maudes-Raedo, Emerging topics in mining software repositories, *Progress in Artif. Intell.* 7 (2018) 237–247, <http://dx.doi.org/10.1007/s13748-018-0147-7>.
- [17] K. Chaturvedi, V. Sing, P. Singh, Tools in mining software repositories, in: 2013 13th International Conference on Computational Science and Its Applications, 2013, pp. 89–98, <http://dx.doi.org/10.1109/ICCSA.2013.22>.
- [18] A. Tripathi, S. Dabral, A. Sureka, University-industry collaboration and open source software (OSS) dataset in mining software repositories (MSR) research, in: 2015 IEEE 1st International Workshop on Software Analytics (SWAN), 2015, pp. 39–40, <http://dx.doi.org/10.1109/SWAN.2015.7070489>.

- [19] A.E. Hassan, The road ahead for Mining Software repositories, in: 2008 Frontiers of Software Maintenance, 2008, pp. 48–57, <http://dx.doi.org/10.1109/FOSM.2008.4659248>.
- [20] G. Vial, Reflections on quality requirements for digital trace data in IS research, Perspectives on Numerical Data Quality in IS Research, Decis. Support Syst. 126 (2019) 113133, <http://dx.doi.org/10.1016/j.dss.2019.113133>, URL: <http://www.sciencedirect.com/science/article/pii/S0167923619301629>.
- [21] B. Kitchenham, Procedures for performing systematic reviews, Keele, UK, Keele University 33 (2004) 1–26.
- [22] K. Petersen, S. Vakkalanka, L. Kuzniarz, Guidelines for conducting systematic mapping studies in software engineering: An update, Inf. Softw. Technol. 64 (2015) 1–18, <http://dx.doi.org/10.1016/j.infsof.2015.03.007>, URL: <http://www.sciencedirect.com/science/article/pii/S0950584915000646>.
- [23] W. Shang, B. Adams, A.E. Hassan, Using Pig as a data preparation language for large-scale mining software repositories studies: An experience report, J. Syst. Softw. 85 (2012) 2195–2204, <http://dx.doi.org/10.1016/j.jss.2011.07.034>, URL: <http://www.sciencedirect.com/science/article/pii/S0164121211002007>.
- [24] M. D'Ambros, R. Robbes, Effective mining of software repositories, in: 2011 27th IEEE International Conference on Software Maintenance (ICSM), 2011, pp. 598–598, <http://dx.doi.org/10.1109/ICSM.2011.6080839>, ISSN: 1063-6773.
- [25] I. Garcia, C. Pacheco, F. Méndez, J.A. Calvo-Manzano, The effects of game-based learning in the acquisition of “soft skills” on undergraduate software engineering courses: A systematic literature review, Comput. Appl. Eng. Edu. 28 (2020) 1327–1354, <http://dx.doi.org/10.1002/cae.22304>.
- [26] M. Abuhamad, J.-s. Rhim, T. AbuHmed, S. Ullah, S. Kang, D. Nyang, Code authorship identification using convolutional neural networks, Future Gener. Comput. Syst. 95 (2019) 104–115, <http://dx.doi.org/10.1016/j.future.2018.12.038>, URL: <http://www.sciencedirect.com/science/article/pii/S0167739X18315528>.
- [27] M.H. Asyrofi, F. Thung, D. Lo, L. Jiang, AUSEarch: Accurate API usage search in GitHub repositories with type resolution, in: 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), 2020, pp. 637–641, <http://dx.doi.org/10.1109/SANER48275.2020.9054809>, ISSN: 1534-5351.
- [28] N.S.A.A. Bakar, Using language-based search in mining large software repositories, Procedia - Soc. Behav. Sci. 27 (2011) 160–168, <http://dx.doi.org/10.1016/j.sbspro.2011.10.594>, URL: <http://www.sciencedirect.com/science/article/pii/S1877042811024219>.
- [29] S. Banerjee, Z. Syed, J. Helmick, M. Culp, K. Ryan, B. Cukic, Automated triaging of very large bug repositories, Inf. Softw. Technol. 89 (2017) 1–13, <http://dx.doi.org/10.1016/j.infsof.2016.09.006>, URL: <http://www.sciencedirect.com/science/article/pii/S0950584916301653>.
- [30] N.A. Batista, M.A. Brandão, G.B. Alves, A.P.C. da Silva, M.M. Moro, Collaboration strength metrics and analyses on GitHub, in: Proceedings of the International Conference on Web Intelligence, in: WI '17, Association for Computing Machinery, Leipzig, Germany, 2017, pp. 170–178, <http://dx.doi.org/10.1145/3106426.3106480>.
- [31] A. Capiluppi, N. Ajiienka, Lexical content as a cooperation aide: A study based on Java software, J. Syst. Softw. 164 (2020) 110543, <http://dx.doi.org/10.1016/j.jss.2020.110543>, URL: <http://www.sciencedirect.com/science/article/pii/S016412122030025X>.
- [32] C.Y. Chong, S.P. Lee, Can commit change history reveal potential fault prone classes? A study on GitHub repositories, in: M. van Sinderen, L.A. Maciaszek (Eds.), Software Technologies, in: Communications in Computer and Information Science, Springer International Publishing, Cham, 2019, pp. 266–281, http://dx.doi.org/10.1007/978-3-030-29157-0_12.
- [33] J. Coelho, M.T. Valente, L.L. Silva, E. Shihab, Identifying unmaintained projects in github, in: Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, in: ESEM '18, Association for Computing Machinery, Oulu, Finland, 2018, pp. 1–10, <http://dx.doi.org/10.1145/3239235.3240501>.
- [34] E. Cohen, M.P. Consens, Large-scale analysis of the co-commit patterns of the active developers in GitHub's top repositories, in: 2018 IEEE/ACM 15th International Conference on Mining Software Repositories (MSR), 2018, pp. 426–436, ISSN: 2574-3864.
- [35] A. Decan, E. Constantinou, T. Mens, H. Rocha, GAP: Forecasting commit activity in git projects, J. Syst. Softw. 165 (2020) 110573, <http://dx.doi.org/10.1016/j.jss.2020.110573>, URL: <http://www.sciencedirect.com/science/article/pii/S0164121220300546>.
- [36] A. Decan, T. Mens, M. Claes, P. Grosjean, When GitHub meets CRAN: An analysis of inter-repository package dependency problems, in: 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER), vol. 1, 2016, pp. 493–504, <http://dx.doi.org/10.1109/SANER.2016.12>.
- [37] M. El Mezouar, F. Zhang, Y. Zou, An empirical study on the teams structures in social coding using GitHub projects, Empir. Softw. Eng. 24 (2019) 3790–3823, <http://dx.doi.org/10.1007/s10664-019-09700-1>.
- [38] G. Farah, D. Correal, Analysis of intercrossed open-source software repositories data in GitHub, in: 2013 8th Computing Colombian Conference (8CCC), 2013, pp. 1–6, <http://dx.doi.org/10.1109/ColombianCC.2013.6637537>.
- [39] B. Gelman, B. Obayomi, J. Moore, D. Slater, Source code analysis dataset, Data in Brief 27 (2019) 104712, <http://dx.doi.org/10.1016/j.dib.2019.104712>, URL: <http://www.sciencedirect.com/science/article/pii/S2352340919310674>.
- [40] M. Gupta, Nirikshan: process mining software repositories to identify inefficiencies, imperfections, and enhance existing process capabilities, in: Companion Proceedings of the 36th International Conference on Software Engineering, in: ICSE Companion 2014, Association for Computing Machinery, Hyderabad, India, 2014, pp. 658–661, <http://dx.doi.org/10.1145/2591062.2591080>.
- [41] F. Hassan, X. Wang, Mining readme files to support automatic building of java projects in software repositories, in: 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C), 2017, pp. 277–279, <http://dx.doi.org/10.1109/ICSE-C.2017.114>.
- [42] Y. Higo, S. Hayashi, S. Kusumoto, On tracking Java methods with Git mechanisms, J. Syst. Softw. 165 (2020) 110571, <http://dx.doi.org/10.1016/j.jss.2020.110571>, URL: <http://www.sciencedirect.com/science/article/pii/S0164121220300522>.
- [43] J. Härtel, M. Heinz, R. Lämmel, EMF patterns of usage on GitHub, in: A. Pierantonio, S. Trujillo (Eds.), Modelling Foundations and Applications, in: Lecture Notes in Computer Science, Springer International Publishing, Cham, 2018, pp. 216–234, http://dx.doi.org/10.1007/978-3-319-92997-2_14.
- [44] S.D. Joshi, S. Chimalakonda, RapidRelease - A dataset of projects and issues on Github with rapid releases, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), 2019, pp. 587–591, <http://dx.doi.org/10.1109/MSR.2019.00088>, ISSN: 2574-3864.
- [45] S. Kawaguchi, P.K. Garg, M. Matsushita, K. Inoue, MUDABlue: An automatic categorization system for Open Source repositories, J. Syst. Softw. 79 (2006) 939–953, <http://dx.doi.org/10.1016/j.jss.2005.06.044>, URL: <http://www.sciencedirect.com/science/article/pii/S0164121205001822>.
- [46] I. Keivanloo, C. Forbes, A. Hmood, M. Erfani, C. Neal, G. Peristerakis, J. Rilling, A Linked Data platform for mining software repositories, in: 2012 9th IEEE Working Conference on Mining Software Repositories (MSR), 2012, pp. 32–35, <http://dx.doi.org/10.1109/MSR.2012.6224296>, ISSN: 2160-1860.
- [47] M. Kiehn, X. Pan, F. Camci, Empirical study in using version histories for change risk classification, in: Proceedings of the 16th International Conference on Mining Software Repositories, in: MSR '19, IEEE Press, Montreal, Quebec, Canada, 2019, pp. 58–62, <http://dx.doi.org/10.1109/MSR.2019.00018>.
- [48] R. Kikas, M. Dumas, D. Pfahl, Issue dynamics in Github projects, in: P. Abrahamsson, L. Corral, M. Oivo, B. Russo (Eds.), Product-Focused Software Process Improvement, in: Lecture Notes in Computer Science, Springer International Publishing, Cham, 2015, pp. 295–310, http://dx.doi.org/10.1007/978-3-319-26844-6_22.
- [49] J. Maqsood, I. Eshraghi, S.S. Ali, Success or failure identification for GitHub's open source projects, in: Proceedings of the 2017 International Conference on Management Engineering, Software Engineering and Service Sciences, in: ICMSS '17, Association for Computing Machinery, Wuhan, China, 2017, pp. 145–150, <http://dx.doi.org/10.1145/3034950.3034957>.
- [50] M. Martinez, M. Monperrus, Mining software repair models for reasoning on the search space of automated program fixing, Empir. Softw. Eng. 20 (2015) 176–205, <http://dx.doi.org/10.1007/s10664-013-9282-8>.
- [51] N. Munaiah, S. Kroh, C. Cabrey, M. Nagappan, Curating GitHub for engineered software projects, Empir. Softw. Eng. 22 (2017) 3219–3253, <http://dx.doi.org/10.1007/s10664-017-9512-6>.
- [52] W. Muylaert, C. De Roover, Prevalence of botched code integrations, in: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), 2017, pp. 503–506, <http://dx.doi.org/10.1109/MSR.2017.40>.
- [53] K.W. Nafi, B. Roy, C.K. Roy, K.A. Schneider, A universal cross language software similarity detector for open source software categorization, J. Syst. Softw. 162 (2020) 110491, <http://dx.doi.org/10.1016/j.jss.2019.110491>, URL: <http://www.sciencedirect.com/science/article/pii/S0164121219302651>.
- [54] P.T. Nguyen, J. Di Rocco, R. Rubel, D. Di Ruscio, CrossSim: Exploiting mutual relationships to detect similar OSS projects, in: 2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), 2018, pp. 388–395, <http://dx.doi.org/10.1109/SEAA.2018.00069>.
- [55] A. Parashar, J.K. Chhabra, Mining software change data stream to predict changeability of classes of object-oriented software system, Evol. Syst. 7 (2016) 117–128, <http://dx.doi.org/10.1007/s12530-016-9151-y>.
- [56] M.M. Rahman, C.K. Roy, An insight into the pull requests of GitHub, in: Proceedings of the 11th Working Conference on Mining Software Repositories, in: MSR 2014, Association for Computing Machinery, Hyderabad, India, 2014, pp. 364–367, <http://dx.doi.org/10.1145/2597073.2597121>.

- [57] M.A. Saied, A. Ouni, H. Sahraoui, R.G. Kula, K. Inoue, D. Lo, Improving reusability of software libraries through usage pattern mining, *J. Syst. Softw.* 145 (2018) 164–179, <http://dx.doi.org/10.1016/j.jss.2018.08.032>, URL: <http://www.sciencedirect.com/science/article/pii/S0164121218301699>.
- [58] A. Santos, M.c. Souza, J. Oliveira, E. Figueiredo, Mining software repositories to identify library experts, in: Proceedings of the VII Brazilian Symposium on Software Components, Architectures, and Reuse, in: SBCARS '18, Association for Computing Machinery, Sao Carlos, Brazil, 2018, pp. 83–91, <http://dx.doi.org/10.1145/3267183.3267192>.
- [59] L.B.L. de Souza, M. de Almeida Maia, Do software categories impact coupling metrics? in: 2013 10th Working Conference on Mining Software Repositories (MSR), 2013, pp. 217–220, <http://dx.doi.org/10.1109/MSR.2013.6624030>, ISSN: 2160-1860.
- [60] G. de la Torre, R. Robbes, A. Bergel, Imprecisions diagnostic in source code deltas, in: Proceedings of the 15th International Conference on Mining Software Repositories, in: MSR '18, Association for Computing Machinery, Gothenburg, Sweden, 2018, pp. 492–502, <http://dx.doi.org/10.1145/3196398.3196404>.
- [61] C. Vendome, A large scale study of license usage on GitHub, in: Proceedings of the 37th International Conference on Software Engineering - Volume 2, in: ICSE '15, IEEE Press, Florence, Italy, 2015, pp. 772–774.
- [62] M. White, C. Vendome, M. Linares-Vasquez, D. Poshvanyk, Toward deep learning software repositories, in: 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, 2015, pp. 334–345, <http://dx.doi.org/10.1109/MSR.2015.38>, ISSN: 2160-1860.
- [63] Y. Yu, Z. Li, G. Yin, T. Wang, H. Wang, A dataset of duplicate pull-requests in github, in: Proceedings of the 15th International Conference on Mining Software Repositories, in: MSR '18, Association for Computing Machinery, Gothenburg, Sweden, 2018, pp. 22–25, <http://dx.doi.org/10.1145/3196398.3196455>.
- [64] A. Zaidman, B. Van Rompaey, S. Demeyer, A. van Deursen, Mining software repositories to study co-evolution of production test code, in: And Validation 2008 1st International Conference on Software Testing, Verification, 2008, pp. 220–229, <http://dx.doi.org/10.1109/ICST.2008.47>, ISSN: 2159-4848.
- [65] Y. Zhang, D. Lo, P.S. Kochhar, X. Xia, Q. Li, J. Sun, Detecting similar repositories on GitHub, in: 2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER), 2017, pp. 13–23, <http://dx.doi.org/10.1109/SANER.2017.7884605>.
- [66] W. Zou, J. Xuan, X. Xie, Z. Chen, B. Xu, How does code style inconsistency affect pull request integration? An exploratory study on 117 GitHub projects, *Empir. Softw. Eng.* 24 (2019) 3871–3903, <http://dx.doi.org/10.1007/s10664-019-09720-x>.
- [67] R. Bana, A. Arora, Influence indexing of developers, repositories, technologies and programming languages on social coding community GitHub, in: 2018 Eleventh International Conference on Contemporary Computing (IC3), 2018, pp. 1–6, <http://dx.doi.org/10.1109/IC3.2018.8530644>, ISSN: 2572-6129.
- [68] H. Borges, A. Hora, M.T. Valente, Understanding the factors that impact the popularity of GitHub repositories, in: 2016 IEEE International Conference on Software Maintenance and Evolution (ICSME), 2016, pp. 334–344, <http://dx.doi.org/10.1109/ICSME.2016.31>.
- [69] H. Borges, M. Tulio Valente, What's in a GitHub star? Understanding repository starring practices in a social coding platform, *J. Syst. Softw.* 146 (2018) 112–129, <http://dx.doi.org/10.1016/j.jss.2018.09.016>, URL: <http://www.sciencedirect.com/science/article/pii/S0164121218301961>.
- [70] N.C. Borle, M. Fegghi, E. Stroulia, R. Greiner, A. Hindle, Analyzing the effects of test driven development in GitHub, *Empir. Softw. Eng.* 23 (2018) 1931–1958, <http://dx.doi.org/10.1007/s10664-017-9576-3>.
- [71] F. Chatziasimidis, I. Stamelos, Data collection and analysis of GitHub repositories and users, in: 2015 6th International Conference on Information, Intelligence, Systems and Applications (IISA), 2015, pp. 1–6, <http://dx.doi.org/10.1109/IISA.2015.7388026>.
- [72] J. Cito, G. Schermann, J.E. Wittern, P. Leitner, S. Zumberi, H.C. Gall, An empirical analysis of the docker container ecosystem on GitHub, in: Proceedings of the 14th International Conference on Mining Software Repositories, in: MSR '17, IEEE Press, Buenos Aires, Argentina, 2017, pp. 323–333, <http://dx.doi.org/10.1109/MSR.2017.67>.
- [73] A. Goyal, N. Sardana, Performance assessment of bug fixing process in open source repositories, *Procedia Comput. Sci.* 167 (2020) 2070–2079, <http://dx.doi.org/10.1016/j.procs.2020.03.247>, URL: <http://www.sciencedirect.com/science/article/pii/S1877050920307134>.
- [74] R. Guidotti, J. Soldani, D. Neri, A. Brogi, Explaining successful docker images using pattern mining analysis, in: M. Mazzara, I. Ober, G. Salaün (Eds.), Software Technologies: Applications and Foundations, in: Lecture Notes in Computer Science, Springer International Publishing, Cham, 2018, pp. 98–113, http://dx.doi.org/10.1007/978-3-030-04771-9_9.
- [75] N. Hajiakhoond Bidoki, G. Sukthankar, H. Keathley, I. Garibay, A cross-repository model for predicting popularity in GitHub, in: 2018 International Conference on Computational Science and Computational Intelligence (CSCI), 2018, pp. 1248–1253, <http://dx.doi.org/10.1109/CSCI46756.2018.00241>.
- [76] J. Jiang, D. Lo, J. He, X. Xia, P.S. Kochhar, L. Zhang, Why and how developers fork what from whom in GitHub, *Empir. Softw. Eng.* 22 (2017) 547–578, <http://dx.doi.org/10.1007/s10664-016-9436-6>.
- [77] D. Kavalier, P. Devanbu, V. Filkov, Whom are you going to call? determinants of @-mentions in Github discussions, *Empir. Softw. Eng.* 24 (2019) 3904–3932, <http://dx.doi.org/10.1007/s10664-019-09728-3>.
- [78] R. Kikas, M. Dumas, D. Pfahl, Using dynamic and contextual features to predict issue lifetime in GitHub projects, in: Proceedings of the 13th International Conference on Mining Software Repositories, in: MSR '16, Association for Computing Machinery, Austin, Texas, 2016, pp. 291–302, <http://dx.doi.org/10.1145/2901739.2901751>.
- [79] S. Lee, H. Baek, J. Jahng, Governance strategies for open collaboration: Focusing on resource allocation in open source software development organizations, *Int. J. Inf. Manage.* 37 (2017) 431–437, <http://dx.doi.org/10.1016/j.ijinfomgt.2017.05.006>, URL: <http://www.sciencedirect.com/science/article/pii/S0268401216308623>.
- [80] N. Li, Z. Li, L. Zhang, Mining frequent patterns from software defect repositories for black-box testing, in: 2010 2nd International Workshop on Intelligent Systems and Applications, 2010, pp. 1–4, <http://dx.doi.org/10.1109/TWISA.2010.5473578>.
- [81] M. Ozer, A. Sapienza, A. Abeliuk, G. Muric, E. Ferrara, Discovering patterns of online popularity from time series, *Expert Syst. Appl.* 151 (2020) 113337, <http://dx.doi.org/10.1016/j.eswa.2020.113337>, URL: <http://www.sciencedirect.com/science/article/pii/S0957417420301627>.
- [82] G. Peng, Co-membership, networks ties, and knowledge flow: An empirical investigation controlling for alternative mechanisms, *Decis. Support Syst.* 118 (2019) 83–90, <http://dx.doi.org/10.1016/j.dss.2019.01.005>, URL: <http://www.sciencedirect.com/science/article/pii/S0167923619300132>.
- [83] Y. Zhang, F.F. Xu, S. Li, Y. Meng, X. Wang, Q. Li, J. Han, HiGitClass: Keyword-driven hierarchical classification of GitHub repositories, in: 2019 IEEE International Conference on Data Mining (ICDM), 2019, pp. 876–885, <http://dx.doi.org/10.1109/ICDM.2019.00098>, ISSN: 2374-8486.
- [84] K.C. Chatzidimitriou, M.D. Papamichail, T. Diamantopoulos, M. Tsapanos, A.L. Symeonidis, Npm-miner: an infrastructure for measuring the quality of the npm registry, in: Proceedings of the 15th International Conference on Mining Software Repositories, in: MSR '18, Association for Computing Machinery, Gothenburg, Sweden, 2018, pp. 42–45, <http://dx.doi.org/10.1145/3196398.3196465>.
- [85] M. Goeminne, T. Mens, A comparison of identity merge algorithms for software repositories, *Sci. Comput. Programm.* 78 (2013) 971–986, <http://dx.doi.org/10.1016/j.scico.2011.11.004>, URL: <http://www.sciencedirect.com/science/article/pii/S0167642311002048>.
- [86] P.T. Nguyen, J. Di Rocco, R. Rubei, D. Di Ruscio, An automated approach to assess the similarity of GitHub repositories, *Softw. Qual. J.* 28 (2020) 595–631, <http://dx.doi.org/10.1007/s11219-019-09483-0>.
- [87] R. Souza, C. Chavez, Characterizing verification of bug fixes in two open source IDEs, in: 2012 9th IEEE Working Conference on Mining Software Repositories (MSR), 2012, pp. 70–73, <http://dx.doi.org/10.1109/MSR.2012.6224301>, ISSN: 2160-1860.
- [88] N.M. Tiwari, G. Upadhyaya, H.A. Nguyen, H. Rajan, Candoia: A platform for building and sharing mining software repositories tools as apps, in: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), 2017, pp. 53–63, <http://dx.doi.org/10.1109/MSR.2017.56>.
- [89] C. Zhou, B. Li, X. Sun, Improving software bug-specific named entity recognition with deep neural network, *J. Syst. Softw.* 165 (2020) 110572, <http://dx.doi.org/10.1016/j.jss.2020.110572>, URL: <http://www.sciencedirect.com/science/article/pii/S0164121220300534>.
- [90] Y. Fu, M. Yan, X. Zhang, L. Xu, D. Yang, J.D. Kymer, Automated classification of software change messages by semi-supervised latent Dirichlet allocation, *Inf. Softw. Technol.* 57 (2015) 369–377, <http://dx.doi.org/10.1016/j.infsof.2014.05.017>, URL: <http://www.sciencedirect.com/science/article/pii/S0950584914001347>.
- [91] J.G.M. Mengerink, J. Noten, A. Serebrenik, Empowering OCL research: a large-scale corpus of open-source data from GitHub, *Empir. Softw. Eng.* 24 (2019) 1574–1609, <http://dx.doi.org/10.1007/s10664-018-9641-6>.
- [92] J. Noten, J.G. Mengerink, A. Serebrenik, A data set of OCL expressions on GitHub, in: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), 2017, pp. 531–534, <http://dx.doi.org/10.1109/MSR.2017.52>.
- [93] G. Robles, T. Ho-Quang, R. Hebig, M.R.V. Chaudron, M.A. Fernandez, An extensive dataset of UML models in GitHub, in: Proceedings of the 14th International Conference on Mining Software Repositories, in: MSR '17, IEEE Press, Buenos Aires, Argentina, 2017, pp. 519–522, <http://dx.doi.org/10.1109/MSR.2017.48>.

- [94] G. Schermann, S. Zumberi, J. Cito, Structured information on state and evolution of dockerfiles on GitHub, in: 2018 IEEE/ACM 15th International Conference on Mining Software Repositories (MSR), 2018, pp. 26–29, ISSN: 2574-3864.
- [95] Y. Yan, M. Menarini, W. Griswold, Mining software contracts for software evolution, in: 2014 IEEE International Conference on Software Maintenance and Evolution, 2014, pp. 471–475, <http://dx.doi.org/10.1109/ICSME.2014.76>, ISSN: 1063-6773.
- [96] S. Brisson, E. Noei, K. Lyons, We are family: analyzing communication in GitHub software repositories and their forks, in: 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), 2020, pp. 59–69, <http://dx.doi.org/10.1109/SANER48275.2020.9054834>, ISSN: 1534-5351.
- [97] H. Xia, C. Li, M. Shi, Design of repositories of GitHub recommendation system based on ternary closure and HITS algorithm, in: 2019 IEEE/ACIS 18th International Conference on Computer and Information Science (ICIS), 2019, pp. 1–5, <http://dx.doi.org/10.1109/ICIS46139.2019.8940236>.
- [98] M. Goeminne, M. Claes, T. Mens, A historical dataset for the Gnome ecosystem, in: 2013 10th Working Conference on Mining Software Repositories (MSR), 2013, pp. 225–228, <http://dx.doi.org/10.1109/MSR.2013.6624032>, ISSN: 2160-1860.
- [99] M. Ohira, Y. Kashiwa, Y. Yamatani, H. Yoshiyuki, Y. Maeda, N. Limsettho, K. Fujino, H. Hata, A. Ihara, K. Matsumoto, A dataset of high impact bugs: manually-classified issue reports, in: Proceedings of the 12th Working Conference on Mining Software Repositories, in: MSR '15, IEEE Press, Florence, Italy, 2015, pp. 518–521.
- [100] J.C.S. Santos, M. Mirakhorli, I. Mujhid, W. Zogaan, BUDGET: A tool for supporting software architecture traceability research, in: 2016 13th Working IEEE/IFIP Conference on Software Architecture (WICSA), 2016, pp. 303–306, <http://dx.doi.org/10.1109/WICSA.2016.47>.
- [101] A. Trockman, R. van Tonder, B. Vasilescu, Striking gold in software repositories? An econometric study of cryptocurrencies on GitHub, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), 2019, pp. 181–185, <http://dx.doi.org/10.1109/MSR.2019.00036>, ISSN: 2574-3864.
- [102] H. Chen, Y. Huang, Z. Liu, X. Chen, F. Zhou, X. Luo, Automatically detecting the scopes of source code comments, J. Syst. Softw. 153 (2019) 45–63, <http://dx.doi.org/10.1016/j.jss.2019.03.010>, URL: <http://www.sciencedirect.com/science/article/pii/S016412121930055X>.
- [103] M. Petticrew, H. Roberts, *Systematic Reviews in the Social Sciences: A Practical Guide*, John Wiley & Sons, 2008.
- [104] M. Ramachandran, Software reuse guidelines, SIGSOFT Softw. Eng. Notes 30 (2005) 1–8, <http://dx.doi.org/10.1145/1061874.1061889>.
- [105] M. Ramachandran, Guidelines based software engineering for developing software components, J. Softw. Eng. Appl. 05 (2012) 1–6, <http://dx.doi.org/10.4236/jsea.2012.51001>.
- [106] D.S. Kolovos, N.D. Matragkas, I. Korkontzelos, S. Ananiadou, R.F. Paige, Assessing the use of eclipse MDE technologies in open-source software projects, in: OSS4MDE@MoDELS, 2015, pp. 1–10.
- [107] A. Howard, C. Zhang, E. Horvitz, Addressing bias in machine learning algorithms: A pilot study on emotion recognition for intelligent systems, in: 2017 IEEE Workshop on Advanced Robotics and Its Social Impacts (ARSO), 2017, pp. 1–7.
- [108] T.B. Kristiansen, Erroneous data and drug industry bias can impair machine learning algorithms, BMJ 367 (2019) <http://dx.doi.org/10.1136/bmj.l6042>, URL: <https://www.bmj.com/content/367/bmj.l6042>.
- [109] G. Gousios, D. Spinellis, Mining software engineering data from GitHub, in: 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C), 2017, pp. 501–502.
- [110] E. Mendes, K. Felizardo, C. Wohlin, M. Kalinowski, Search strategy to update systematic literature reviews in software engineering, in: 2019 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), 2019, pp. 355–362, <http://dx.doi.org/10.1109/SEAA.2019.00061>.
- [111] N.K. Nagwani, A. Bhansali, A data mining model to predict software bug complexity using bug estimation and clustering, in: 2010 International Conference on Recent Trends in Information, Telecommunication and Computing, 2010, pp. 13–17, <http://dx.doi.org/10.1109/ITC.2010.56>.
- [112] R. van Tonder, A. Trockman, C.L. Goues, A panel data set of cryptocurrency development activity on GitHub, in: Proceedings of the 16th International Conference on Mining Software Repositories, in: MSR '19, IEEE Press, Montreal, Quebec, Canada, 2019, pp. 186–190, <http://dx.doi.org/10.1109/MSR.2019.00037>.
- [113] E. Kouroshfar, M. Mirakhorli, H. Bagheri, L. Xiao, S. Malek, Y. Cai, A study on the role of software architecture in the evolution and quality of software, in: 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, 2015, pp. 246–257, <http://dx.doi.org/10.1109/MSR.2015.30>, ISSN: 2160-1860.
- [114] Neelofar, M.Y. Javed, H. Mohsin, An automated approach for software bug classification, in: 2012 Sixth International Conference on Complex, Intelligent, and Software Intensive Systems, 2012, pp. 414–419, <http://dx.doi.org/10.1109/CISIS.2012.132>.
- [115] U. Raja, M.J. Tretter, Antecedents of open source software defects: A data mining approach to model formulation, validation and testing, Inform. Technol. Manag. 10 (2009) 235, <http://dx.doi.org/10.1007/s10799-009-0062-5>.
- [116] M. Harman, Y. Jia, Y. Zhang, App store mining and analysis: MSR for app stores, in: 2012 9th IEEE Working Conference on Mining Software Repositories (MSR), 2012, pp. 108–111, <http://dx.doi.org/10.1109/MSR.2012.6224306>, ISSN: 2160-1860.
- [117] B.V.A. Prakash, D.V. Ashoka, V.N.M. Aradhya, Application of data mining techniques for software reuse process, Proc. Technol. 4 (2012) 384–389, <http://dx.doi.org/10.1016/j.procty.2012.05.059>, URL: <http://www.sciencedirect.com/science/article/pii/S2212017312003386>.
- [118] F. Costa, D. de Oliveira, E. Ogasawara, A.A.B. Lima, M. Mattoso, Athena: Text mining based discovery of scientific workflows in disperse repositories, in: Z. Lacroix, M.a.E. Vidal (Eds.), Resource Discovery, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2012, pp. 104–121, http://dx.doi.org/10.1007/978-3-642-27392-6_8.
- [119] A. Ampatzoglou, O. Michou, I. Stamelos, Building and mining a repository of design pattern instances: Practical and research benefits, Entertain. Comput. 4 (2013) 131–142, <http://dx.doi.org/10.1016/j.entcom.2012.10.002>, URL: <http://www.sciencedirect.com/science/article/pii/S1875952112000195>.
- [120] F. Arcelli Fontana, M. Rolla, M. Zanoni, Capturing software evolution and change through code repository smells, in: T. Dingsøyr, N.B. Moe, R. Tonelli, S. Counsell, C. Gencel, K. Petersen (Eds.), Agile Methods. Large-Scale Development, Refactoring, Testing, and Estimation, in: Lecture Notes in Business Information Processing, Springer International Publishing, Cham, 2014, pp. 148–165, http://dx.doi.org/10.1007/978-3-319-14358-3_13.
- [121] G.A.A. Prana, C. Treude, F. Thung, T. Atapattu, D. Lo, Categorizing the content of GitHub README files, Empir. Softw. Eng. 24 (2019) 1296–1327, <http://dx.doi.org/10.1007/s10664-018-9660-3>.
- [122] M. Soll, M. Vosgerau, ClassifyHub: An algorithm to classify GitHub repositories, in: G. Kern-Isberner, J. Fürnkranz, M. Thimm (Eds.), KI 2017: Advances in Artificial Intelligence, in: Lecture Notes in Computer Science, Springer International Publishing, Cham, 2017, pp. 373–379, http://dx.doi.org/10.1007/978-3-319-67190-1_34.
- [123] S. Kim, E.J. Whitehead, Y. Zhang, Classifying software changes: Clean or Buggy? IEEE Trans. Softw. Eng. 34 (2008) 181–196, <http://dx.doi.org/10.1109/TSE.2007.70773>.
- [124] M.-A. Sicilia, E. Garcí a Barriocanal, S. Sánchez-Alonso, Community curation in open dataset repositories: Insights from Zenodo, Procedia Comput. Sci. 106 (2017) 54–60, <http://dx.doi.org/10.1016/j.procs.2017.03.009>, URL: <http://www.sciencedirect.com/science/article/pii/S1877050917302776>.
- [125] L. Madeyski, M. Kawalerowicz, Continuous defect prediction: The idea and a related dataset, in: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), 2017, pp. 515–518, <http://dx.doi.org/10.1109/MSR.2017.46>.
- [126] D. Kolovos, P. Neubauer, K. Barmpis, N. Matragkas, R. Paige, Crossflow: A framework for distributed mining of software repositories, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), 2019, pp. 155–159, <http://dx.doi.org/10.1109/MSR.2019.00032>, ISSN: 2574-3864.
- [127] M. Kumar J., S. Dubey, B. Balaji, D. Rao, D. Rao, Data visualization on GitHub repository parameters using elastic search and Kibana, in: 2018 2nd International Conference on Trends in Electronics and Informatics (ICOEI), 2018, pp. 554–558, <http://dx.doi.org/10.1109/ICOEI.2018.8553755>.
- [128] R. Selby, Enabling reuse-based software development of large-scale systems, IEEE Trans. Softw. Eng. 31 (2005) 495–510, <http://dx.doi.org/10.1109/TSE.2005.69>.
- [129] G. Canfora, L. Cerulo, Fine grained indexing of software repositories to support impact analysis, in: Proceedings of the 2006 International Workshop on Mining Software Repositories, (MSR '06), Association for Computing Machinery, Shanghai, China, 2006, pp. 105–111, <http://dx.doi.org/10.1145/1137983.1138009>.
- [130] B. Vasilescu, D. Posnett, B. Ray, M.G. van den Brand, A. Serebrenik, P. Devanbu, V. Filkov, Gender and tenure diversity in GitHub teams, in: Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems, in: CHI '15, Association for Computing Machinery, Seoul, Republic of Korea, 2015, pp. 3789–3798, <http://dx.doi.org/10.1145/2702123.2702549>.
- [131] A. Lazar, S. Ritchey, B. Sharif, Generating duplicate bug datasets, in: Proceedings of the 11th Working Conference on Mining Software Repositories, in: MSR 2014, Association for Computing Machinery, Hyderabad, India, 2014, pp. 392–395, <http://dx.doi.org/10.1145/2597073.2597128>.

- [132] R.K.-W. Lee, D. Lo, GitHub and stack overflow: Analyzing developer interests across multiple social collaborative platforms, in: G.L. Ciampaglia, A. Mashhadi, T. Yasseri (Eds.), *Social Informatics*, in: *Lecture Notes in Computer Science*, Springer International Publishing, Cham, 2017, pp. 245–256, http://dx.doi.org/10.1007/978-3-319-67256-4_19.
- [133] X. Cai, J. Zhu, B. Shen, Y. Chen, GRETA: Graph-based tag assignment for GitHub repositories, in: 2016 IEEE 40th Annual Computer Software and Applications Conference (COMPSAC), vol. 1, 2016, pp. 63–72, <http://dx.doi.org/10.1109/COMPSAC.2016.124>, ISSN: 0730-3157.
- [134] S.S. Manes, O. Baysal, How often and what StackOverflow posts do developers reference in their GitHub projects? in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), 2019, pp. 235–239, <http://dx.doi.org/10.1109/MSR.2019.00047>, ISSN: 2574-3864.
- [135] C.A. Thompson, G.C. Murphy, M. Palyart, M. Ga' sparic, How software developers use work breakdown relationships in issue repositories, in: 2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR), 2016, pp. 281–285.
- [136] F. Mulder, A. Zaidman, Identifying cross-cutting concerns using software repository mining, in: *Proceedings of the Joint ERCIM Workshop on Software Evolution (EVOL) and International Workshop on Principles of Software Evolution (IWPSE)*, IWPSE-EVOL '10, Association for Computing Machinery, Antwerp, Belgium, 2010, pp. 23–32, <http://dx.doi.org/10.1145/1862372.1862381>.
- [137] J.a.E. Montandon, L.L. Silva, M.T. Valente, Identifying experts in software libraries and frameworks among GitHub users, in: *Proceedings of the 16th International Conference on Mining Software Repositories*, in: MSR '19, IEEE Press, Montreal, Quebec, Canada, 2019, pp. 276–287, <http://dx.doi.org/10.1109/MSR.2019.00054>.
- [138] J. Hayashi, Y. Higo, S. Matsumoto, S. Kusumoto, Impacts of daylight saving time on software development, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), 2019, pp. 502–506, <http://dx.doi.org/10.1109/MSR.2019.00076>, ISSN: 2574-3864.
- [139] Y. Hu, J. Zhang, X. Bai, S. Yu, Z. Yang, Influence analysis of Github repositories, *SpringerPlus* 5 (2016) 1268, <http://dx.doi.org/10.1186/s40064-016-2897-7>.
- [140] C. Hauff, G. Gousios, Matching GitHub developer profiles to job advertisements, in: *Proceedings of the 12th Working Conference on Mining Software Repositories*, in: MSR '15, IEEE Press, Florence, Italy, 2015, pp. 362–366.
- [141] A.S. Badashian, E. Stroulia, Measuring user influence in GitHub: the million follower fallacy, in: *Proceedings of the 3rd International Workshop on CrowdSourcing in Software Engineering*, (CSI-SE '16), Association for Computing Machinery, Austin, Texas, 2016, pp. 15–21, <http://dx.doi.org/10.1145/2897659.2897663>.
- [142] Y. Yu, H. Wang, G. Yin, B. Liu, Mining and recommending software features across multiple web repositories, in: *Proceedings of the 5th Asia-Pacific Symposium on Internetware*, in: *Internetware '13*, Association for Computing Machinery, Changsha, China, 2013, pp. 1–9, <http://dx.doi.org/10.1145/2532443.2532453>.
- [143] T.S. Heinze, V. Stefanko, W. Amme, Mining BPMN processes on GitHub for tool validation and development, in: S. Nurcan, I. Reinhartz-Berger, P. Soffer, J. Zdravkovic (Eds.), *Enterprise, Business-Process and Information Systems Modeling*, in: *Lecture Notes in Business Information Processing*, Springer International Publishing, Cham, 2020, pp. 193–208, http://dx.doi.org/10.1007/978-3-030-49418-6_13.
- [144] P. Abate, R. Di Cosmo, L. Gesbert, F. Le Fessant, R. Treinen, S. Zacchiroli, Mining component repositories for installability issues, in: 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, 2015, pp. 24–33, <http://dx.doi.org/10.1109/MSR.2015.10>, ISSN: 2160-1860.
- [145] L. Yu, S. Ramaswamy, Mining CVS repositories to understand open-source project developer roles, in: *Fourth International Workshop on Mining Software Repositories (MSR'07:ICSE Workshops 2007)*, 2007, pp. 8, <http://dx.doi.org/10.1109/MSR.2007.19>, ISSN: 2160-1860.
- [146] G. Sprint, J. Conci, Mining GitHub classroom commit behavior in elective and introductory computer science courses, *J. Comput. Sci. Colleges* 35 (2019) 76–84.
- [147] Y. Weicheng, S. Beijun, X. Ben, Mining GitHub: Why commit stops – Exploring the relationship between developer's commit pattern and file version evolution, in: 2013 20th Asia-Pacific Software Engineering Conference (APSEC), vol. 2, 2013, pp. 165–169, <http://dx.doi.org/10.1109/APSEC.2013.133>, ISSN: 1530-1362.
- [148] S. Yatish, J. Jiarpakdee, P. Thongtanunam, C. Tantithamthavorn, Mining software defects: Should we consider affected releases? in: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), 2019, pp. 654–665, <http://dx.doi.org/10.1109/ICSE.2019.00075>, ISSN: 1558-1225.
- [149] T. Wang, H. Wang, G. Yin, C.X. Ling, X. Li, P. Zou, Mining software profile across multiple repositories for hierarchical categorization, in: 2013 IEEE International Conference on Software Maintenance, 2013, pp. 240–249, <http://dx.doi.org/10.1109/ICSM.2013.35>, ISSN: 1063-6773.
- [150] X. Meng, B.P. Miller, W.R. Williams, A.R. Bernat, Mining software repositories for accurate authorship, in: 2013 IEEE International Conference on Software Maintenance, 2013, pp. 250–259, <http://dx.doi.org/10.1109/ICSM.2013.36>, ISSN: 1063-6773.
- [151] O. Meqdadi, N. Alhindawi, J. Alsakran, A. Saifan, H. Migdadi, Mining software repositories for adaptive change commits using machine learning techniques, *Inf. Softw. Technol.* 109 (2019) 80–91, <http://dx.doi.org/10.1016/j.infsof.2019.01.008>, URL: <http://www.sciencedirect.com/science/article/pii/S0950584919300084>.
- [152] O. Vandecruys, D. Martens, B. Baesens, C. Mues, M. De Backer, R. Haesen, Mining software repositories for comprehensible software fault prediction models, *J. Syst. Softw.* 81 (2008) 823–839, <http://dx.doi.org/10.1016/j.jss.2007.07.034>, URL: <http://www.sciencedirect.com/science/article/pii/S0164121207001902>.
- [153] H.K. Dam, B.T.R. Savarimuthu, D. Avery, A. Ghose, Mining software repositories for social norms, in: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, vol. 2, 2015, pp. 627–630, <http://dx.doi.org/10.1109/ICSE.2015.209>, ISSN: 1558-1225.
- [154] K. Mierle, K. Laven, S. Roweis, G. Wilson, Mining student CVS repositories for performance indicators, in: *Proceedings of the 2005 International Workshop on Mining Software Repositories*, MSR '05, Association for Computing Machinery, St. Louis, Missouri, 2005, pp. 1–5, <http://dx.doi.org/10.1145/1083142.1083150>.
- [155] J. Wang, Y. Dang, H. Zhang, K. Chen, T. Xie, D. Zhang, Mining succinct and high-coverage API usage patterns from source code, in: 2013 10th Working Conference on Mining Software Repositories (MSR), 2013, pp. 319–328, <http://dx.doi.org/10.1109/MSR.2013.6624045>, ISSN: 2160-1860.
- [156] X. Yang, R.G. Kula, N. Yoshida, H. Iida, Mining the modern code review repositories: A dataset of people, process and product, in: 2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR), 2016, pp. 460–463.
- [157] Y. Ma, H. Li, J. Hu, R. Xie, Y. Chen, Mining the network of the programmers: A data-driven analysis of GitHub, in: *Proceedings of the 12th Chinese Conference on Computer Supported Cooperative Work and Social Computing*, in: *ChineseCSCW '17*, Association for Computing Machinery, Chongqing, China, 2017, pp. 165–168, <http://dx.doi.org/10.1145/3127404.3127431>.
- [158] N.H. Bidoki, M. Schiappa, G. Sukthankar, I. Garibay, Modeling social coding dynamics with sampled historical data, *Online Soc. Netw. Media* 16 (2020) 100070, <http://dx.doi.org/10.1016/j.osnem.2020.100070>, URL: <http://www.sciencedirect.com/science/article/pii/S2468696420300112>.
- [159] X. Sun, B. Li, H. Leung, B. Li, Y. Li, MSR4SM: Using topic models to effectively mining software repositories for software maintenance tasks, *Inf. Softw. Technol.* 66 (2015) 1–12, <http://dx.doi.org/10.1016/j.infsof.2015.05.003>, URL: <http://www.sciencedirect.com/science/article/pii/S0950584915001007>.
- [160] G. Destefanis, M. Ortu, D. Bowes, M. Marchesi, R. Tonelli, On measuring affects of github issues' commenters, in: *Proceedings of the 3rd International Workshop on Emotion Awareness in Software Engineering*, SEmotion '18, Association for Computing Machinery, Gothenburg, Sweden, 2018, pp. 14–19, <http://dx.doi.org/10.1145/3194932.3194936>.
- [161] P. Anbalagan, M. Vouk, On mining data across software repositories, in: 2009 6th IEEE International Working Conference on Mining Software Repositories, 2009, pp. 171–174, <http://dx.doi.org/10.1109/MSR.2009.5069498>, ISSN: 2160-1860.
- [162] K.V.R. Paixão, C.c.Z. Félício, F.M. Delfim, M. De A. Maia, On the interplay between non-functional requirements and builds on continuous integration, in: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), 2017, pp. 479–482, <http://dx.doi.org/10.1109/MSR.2017.33>.
- [163] G. Vale, A. Schmid, A.R. Santos, E.S. de Almeida, S. Apel, On the relation between Github communication activity and merge conflicts, *Empir. Softw. Eng.* 25 (2020) 402–433, <http://dx.doi.org/10.1007/s10664-019-09774-x>.
- [164] T. Zhang, G. Yang, B. Lee, A.T.S. Chan, Predicting severity of bug report by mining bug repository with concept profile, in: *Proceedings of the 30th Annual ACM Symposium on Applied Computing*, in: SAC '15, Association for Computing Machinery, Salamanca, Spain, 2015, pp. 1553–1558, <http://dx.doi.org/10.1145/2695664.2695872>.
- [165] T.G. Habing, J. Eke, J.S. Kaczmarek, Repository software evaluation using the audit checklist for certification of trusted digital repositories, in: *Proceedings of the 6th ACM/IEEE-CS Joint Conference on Digital Libraries (JCDL '06)*, 2006, pp. 107–108, <http://dx.doi.org/10.1145/1141753.1141774>.
- [166] S. Raemaekers, A. van Deursen, J. Visser, Semantic versioning and impact of breaking changes in the Maven repository, *J. Syst. Softw.* 129 (2017) 140–158, <http://dx.doi.org/10.1016/j.jss.2016.04.008>, URL: <http://www.sciencedirect.com/science/article/pii/S0164121216300243>.
- [167] W. Leibzon, Social network of software development at GitHub, in: *Proceedings of the 2016 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*, in: *ASONAM '16*, IEEE Press, Davis, California, 2016, pp. 1374–1376.

- [168] G. Czibula, Z. Marian, I.G. Czibula, Software defect prediction using relational association rule mining, *Inform. Sci.* 264 (2014) 260–278, <http://dx.doi.org/10.1016/j.ins.2013.12.031>, URL: <http://www.sciencedirect.com/science/article/pii/S0020025513008876>.
- [169] A.K. Dwivedi, A. Tirkey, S.K. Rath, Software design pattern mining using classification-based techniques, *Front. Comput. Sci.* 12 (2018) 908–922, <http://dx.doi.org/10.1007/s11704-017-6424-y>.
- [170] E. Linstead, S. Bajracharya, T. Ngo, P. Rigor, C. Lopes, P. Baldi, Sourcerer: mining and searching internet-scale software repositories, *Data Min. Knowl. Discov.* 18 (2009) 300–336, <http://dx.doi.org/10.1007/s10618-008-0118-x>.
- [171] O. Mizuno, S. Ikami, S. Nakaichi, T. Kikuno, Spam filter based approach for finding fault-prone software modules, in: Fourth International Workshop on Mining Software Repositories (MSR'07:ICSE Workshops 2007), 2007, pp. 4, <http://dx.doi.org/10.1109/MSR.2007.29>, ISSN: 2160-1860.
- [172] M. Ortu, A. Murgia, G. Destefanis, P. Tourani, R. Tonelli, M. Marchesi, B. Adams, The emotional side of software developers in JIRA, in: 2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR), 2016, pp. 480–483.
- [173] A. Lamkanfi, J. Pérez, S. Demeyer, The Eclipse and Mozilla defect tracking dataset: A genuine dataset for mining bug information, in: 2013 10th Working Conference on Mining Software Repositories (MSR), 2013, pp. 203–206, <http://dx.doi.org/10.1109/MSR.2013.6624028>, ISSN: 2160-1860.
- [174] S. Raemaekers, A. van Deursen, J. Visser, The Maven repository dataset of metrics, changes, and dependencies, in: 2013 10th Working Conference on Mining Software Repositories (MSR), 2013, pp. 221–224, <http://dx.doi.org/10.1109/MSR.2013.6624031>, ISSN: 2160-1860.
- [175] S.S. Alqahtani, E.E. Eghan, J. Rilling, Tracing known security vulnerabilities in software repositories – A Semantic Web enabled modeling approach, *Sci. Comput. Programm.* 121 (2016) 153–175, <http://dx.doi.org/10.1016/j.scico.2016.01.005>, URL: <http://www.sciencedirect.com/science/article/pii/S0167642316000253>.
- [176] I. Neamtii, J.S. Foster, M. Hicks, Understanding source code evolution using abstract syntax tree matching, in: Proceedings of the 2005 International Workshop on Mining Software Repositories, MSR '05, Association for Computing Machinery, St. Louis, Missouri, 2005, pp. 1–5, <http://dx.doi.org/10.1145/1083142.1083143>.
- [177] D.M. German, Using software distributions to understand the relationship among free and open source software projects, in: Proceedings of the Fourth International Workshop on Mining Software Repositories, MSR '07, IEEE Computer Society, USA, 2007, pp. 24, <http://dx.doi.org/10.1109/MSR.2007.32>.
- [178] P. Weissgerber, M. Pohl, M. Burch, Visual data mining in software archives to detect how developers work together, in: Proceedings of the Fourth International Workshop on Mining Software Repositories, MSR '07, IEEE Computer Society, USA, 2007, pp. 9, <http://dx.doi.org/10.1109/MSR.2007.34>.